

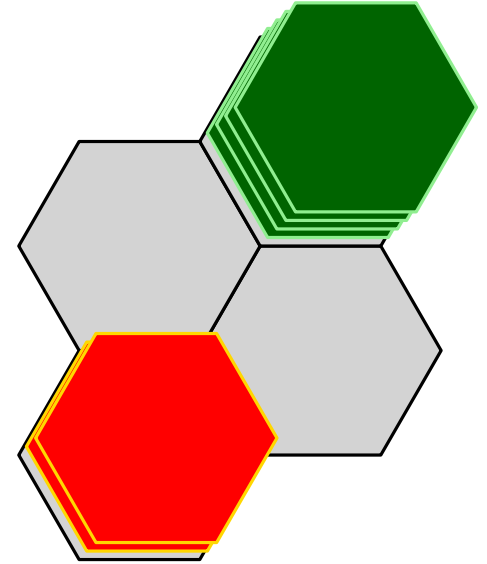
Hexasort

The Complexity of Stacking Colors on Graphs^{*}

Linus Klocker and Simon Dominik Fink

21.05.2026 · Session 9

13th International Conference on Fun with Algorithms 2026

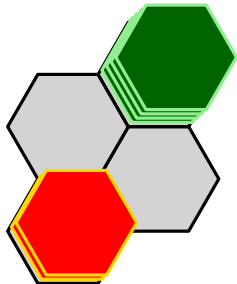


ALGORITHMS AND
COMPLEXITY GROUP

^{*} WWTF grant [10.47379/ICT22029] and FWF grant [10.55776/Y1329]

Introduction – Popular Games

Many popular puzzle games become NP-hard when generalized.

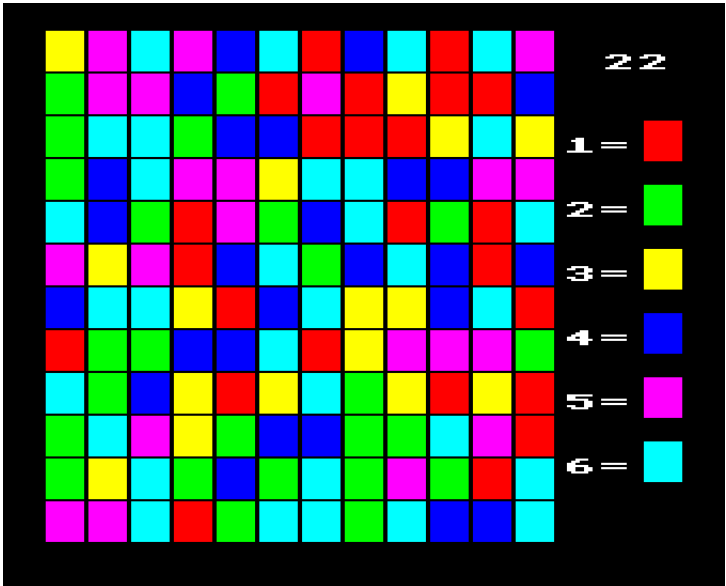


5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Sudoku

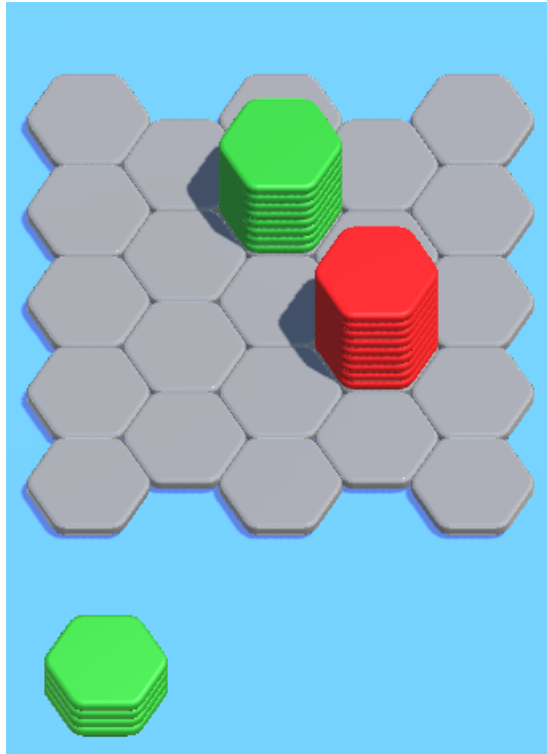
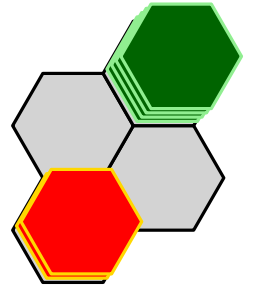


Candy Crush



Flood-It

Introduction – What is Hexasort?

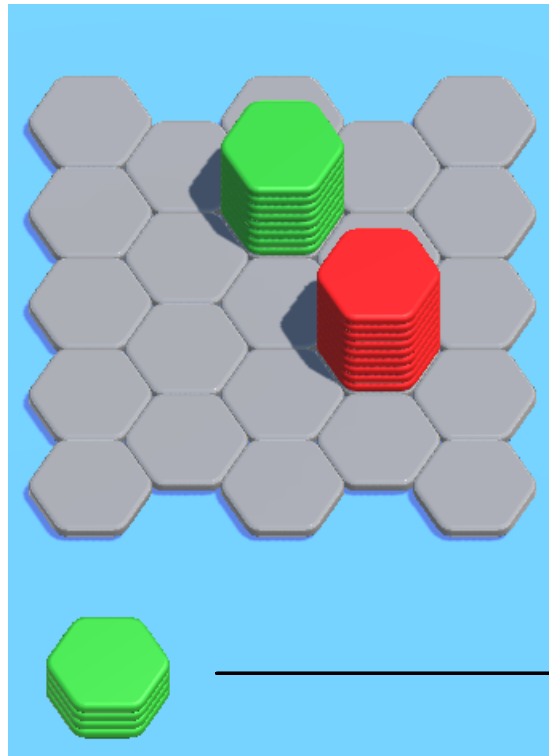


www.spielaffe.de/Spiel/Hexa-Sort-3D

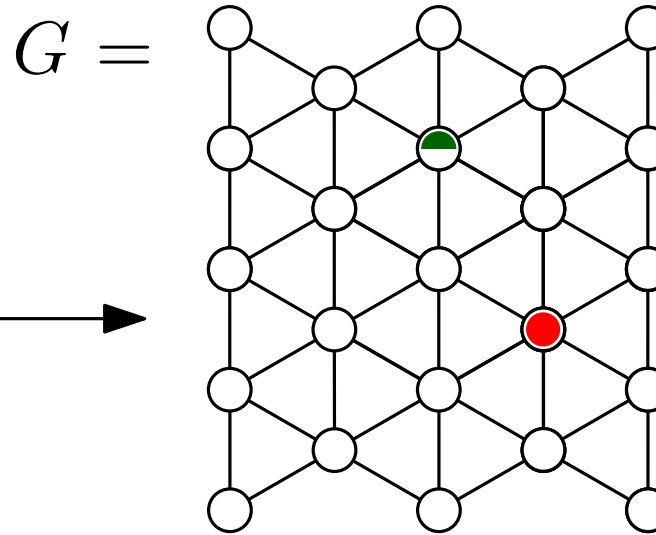
- Place stack
- Adjacent equal colored stacks merge
- If threshold reached, the merged stack vanishes
- Objective: place the whole sequence

Introduction – Our formal model

- An instance (G, S, t) of Hexasort consists of

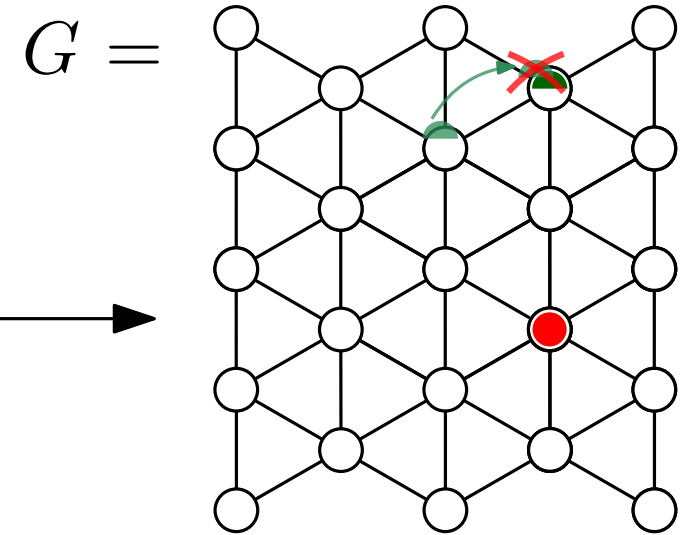


$t = 10$



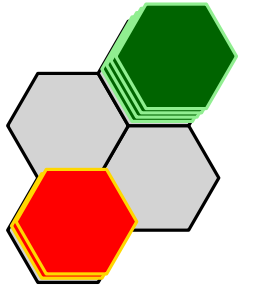
$$S = \{(\text{green}, 4)\}$$

$t = 10$



$$S = \{\}$$

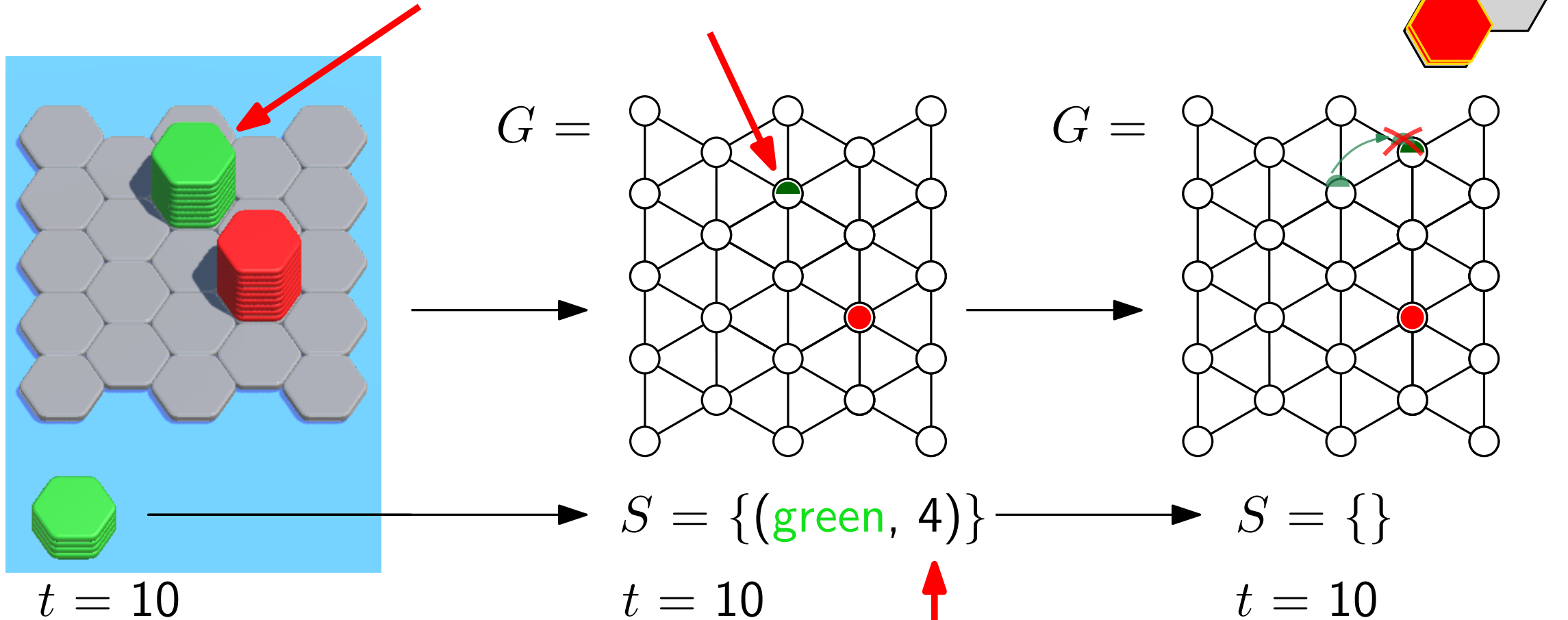
$t = 10$



- **Fitting:** ✓
- **Empty:** ✗

Introduction – Our formal model

- An instance (G, S, t) of Hexasort consists of

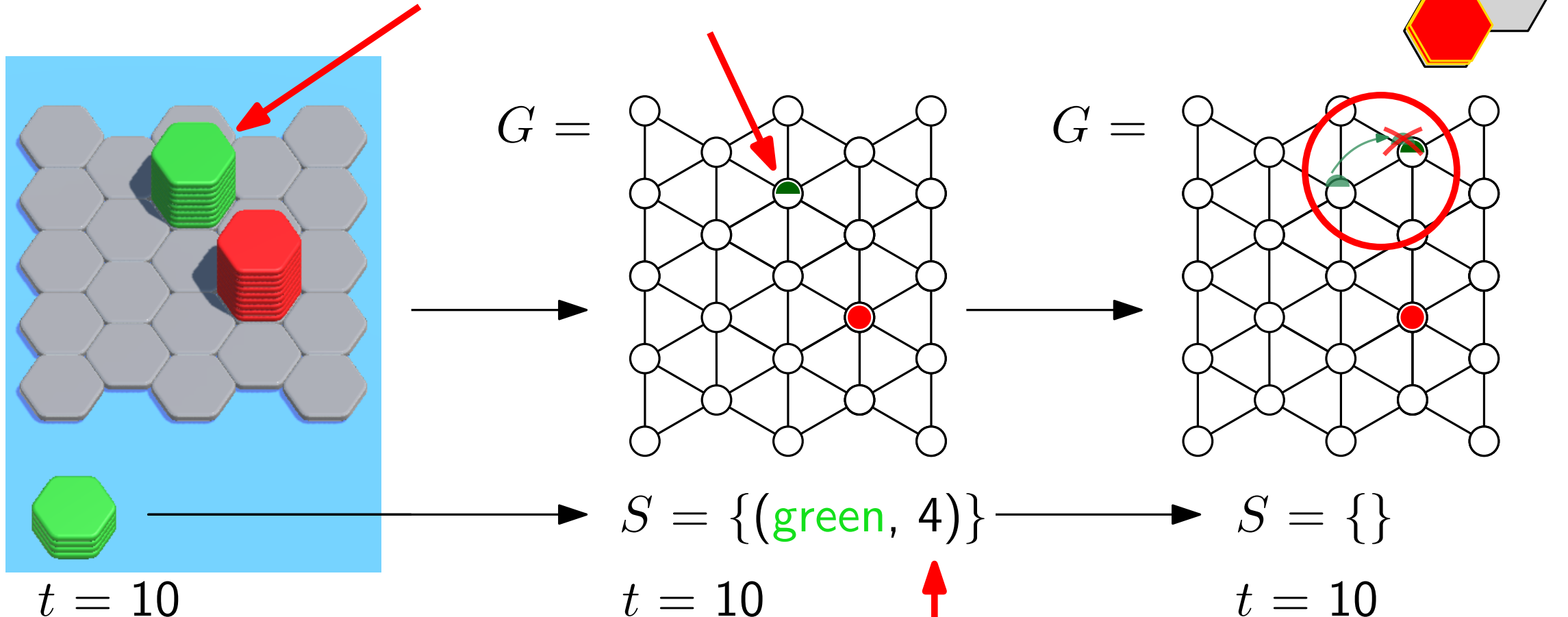


- Omit stack heights in graphs

- **Fitting:** ✓
- **Empty:** ✗

Introduction – Our formal model

- An instance (G, S, t) of Hexasort consists of



- Omit stack heights in graphs
- Merge always onto newly placed stack

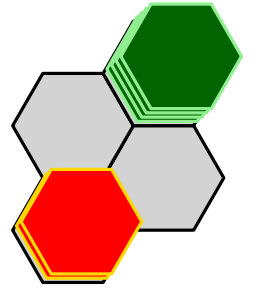
- **Fitting:** ✓
- **Empty:** ✗

Introduction – Our formal model

We consider the Hexasort problem in two variants:

Fitting Hexasort

Input: A Hexasort instance (G, S, t) consisting of



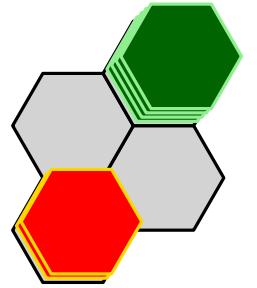
Introduction – Our formal model

We consider the Hexasort problem in two variants:

Fitting Hexasort

Input: A Hexasort instance (G, S, t) consisting of

- an undirected graph $G = (V, E) = \text{board}$



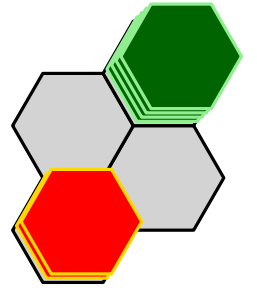
Introduction – Our formal model

We consider the Hexasort problem in two variants:

Fitting Hexasort

Input: A Hexasort instance (G, S, t) consisting of

- an undirected graph $G = (V, E) = \text{board}$
- a sequence S of monochromatic stacks $s = (c, h)$ of color c and height h



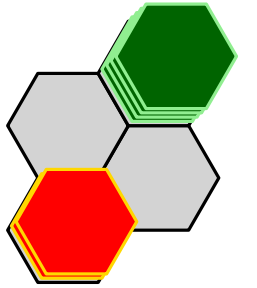
Introduction – Our formal model

We consider the Hexasort problem in two variants:

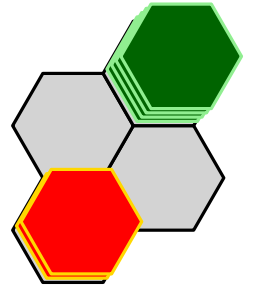
Fitting Hexasort

Input: A Hexasort instance (G, S, t) consisting of

- an undirected graph $G = (V, E) = \text{board}$
- a sequence S of monochromatic stacks $s = (c, h)$ of color c and height h
- a threshold $t \in \mathbb{N}^+$



Introduction – Our formal model



We consider the Hexasort problem in two variants:

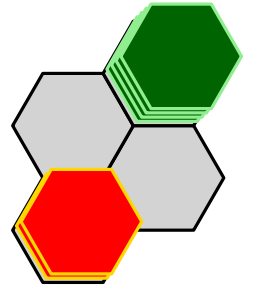
Fitting Hexasort

Input: A Hexasort instance (G, S, t) consisting of

- an undirected graph $G = (V, E) = \text{board}$
- a sequence S of monochromatic stacks $s = (c, h)$ of color c and height h
- a threshold $t \in \mathbb{N}^+$

Task: Can we place all stacks without running out of board space?

Introduction – Our formal model



We consider the Hexasort problem in two variants:

Fitting Hexasort

Input: A Hexasort instance (G, S, t) consisting of

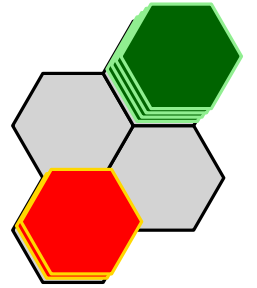
- an undirected graph $G = (V, E) = \text{board}$
- a sequence S of monochromatic stacks $s = (c, h)$ of color c and height h
- a threshold $t \in \mathbb{N}^+$

Task: Can we place all stacks without running out of board space?

Empty Hexasort

Input: A Hexasort instance (G, S, t)

Introduction – Our formal model



We consider the Hexasort problem in two variants:

Fitting Hexasort

Input: A Hexasort instance (G, S, t) consisting of

- an undirected graph $G = (V, E) = \text{board}$
- a sequence S of monochromatic stacks $s = (c, h)$ of color c and height h
- a threshold $t \in \mathbb{N}^+$

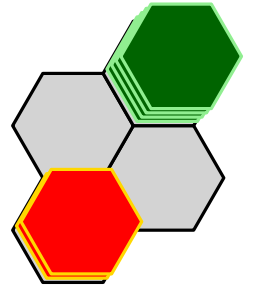
Task: Can we place all stacks without running out of board space?

Empty Hexasort

Input: A Hexasort instance (G, S, t)

Task: Can we place all stacks without running out of board space and end with an empty board?

Introduction – Our formal model



We consider the Hexasort problem in two variants:

Fitting Hexasort

Input: A Hexasort instance (G, S, t) consisting of

- an undirected graph $G = (V, E) = \text{board}$
- a sequence S of monochromatic stacks $s = (c, h)$ of color c and height h
- a threshold $t \in \mathbb{N}^+$

Task: Can we place all stacks without running out of board space?

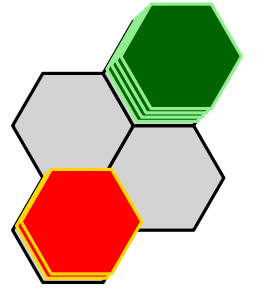
Empty Hexasort

Input: A Hexasort instance (G, S, t)

Task: Can we place all stacks without running out of board space and end with an empty board?

No complications from the game such as non-empty starting board or stacks with multiple colors are considered.

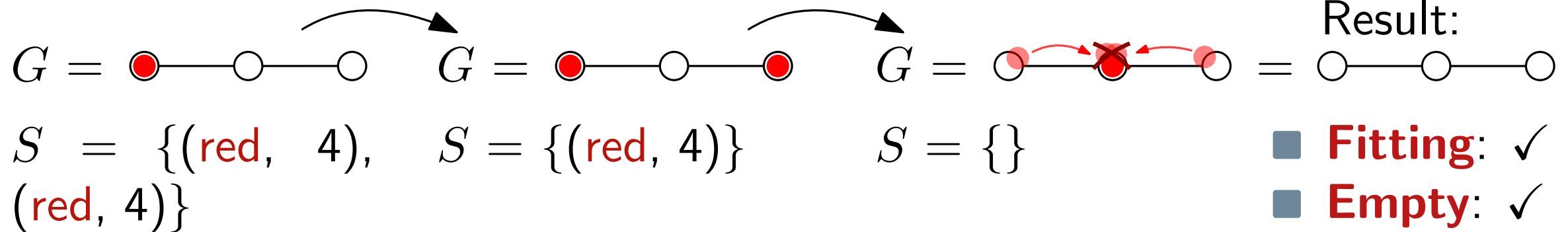
Introduction – Reduction from Empty to Fitting



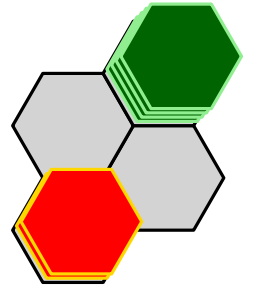
- An instance (G, S, t) of Hexasort consists of

$$G = \bigcirc - \bigcirc - \bigcirc \quad S = \{(\text{red}, 4), (\text{red}, 4), (\text{red}, 4)\} \quad t = 5$$

- Playthrough:



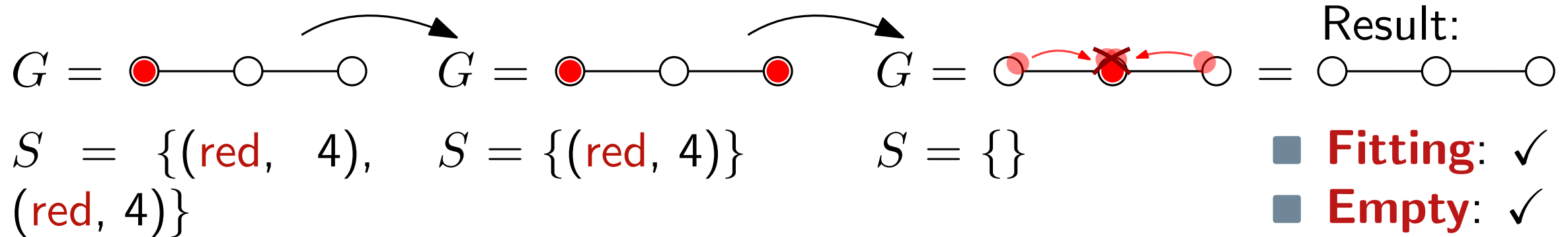
Introduction – Reduction from Empty to Fitting



- An instance (G, S, t) of Hexasort consists of

$$G = \bigcirc - \bigcirc - \bigcirc \quad S = \{(\text{red}, 4), (\text{red}, 4), (\text{red}, 4)\} \quad t = 5$$

- Playthrough:



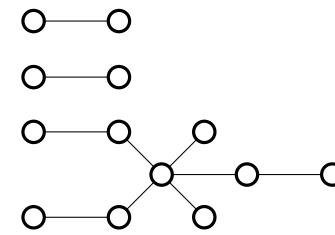
- Empty $\leq$ Fitting: append $|V|$ stacks to S , each with a new distinct color and height $< t$.

$$S = \{(\text{red}, 4), (\text{red}, 4), (\text{red}, 4), (\text{green}, 1), (\text{blue}, 1), (\text{yellow}, 1)\}$$

$$G = \text{green} - \text{blue} - \text{yellow}$$

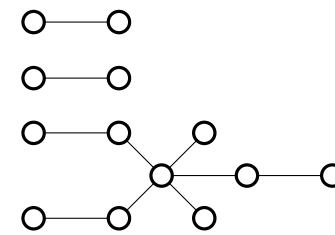
Hardness on Empty Hexasort:

- weak NP-hardness on two disjoint edges
- weak NP-hardness on a spider graph



Hardness on Empty Hexasort:

- weak NP-hardness on two disjoint edges
- weak NP-hardness on a spider graph



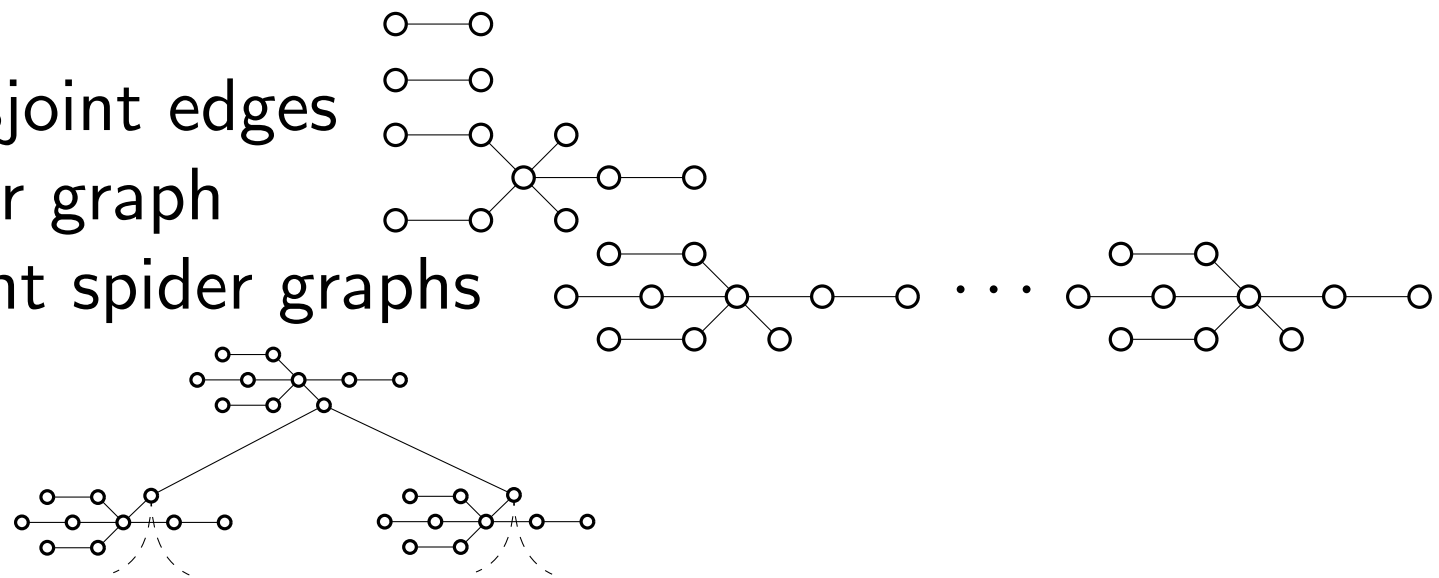
Recap on Weak NP-hardness:

Weak NP-hardness describes a set of problems which are NP-hard in general, but are in P if every number in the input is polynomially bounded by the input size.

Intuitive Understanding of Weak NP-hardness: The problem is only hard when the input contains exponentially large numbers, if the numbers are small the problem is easy.

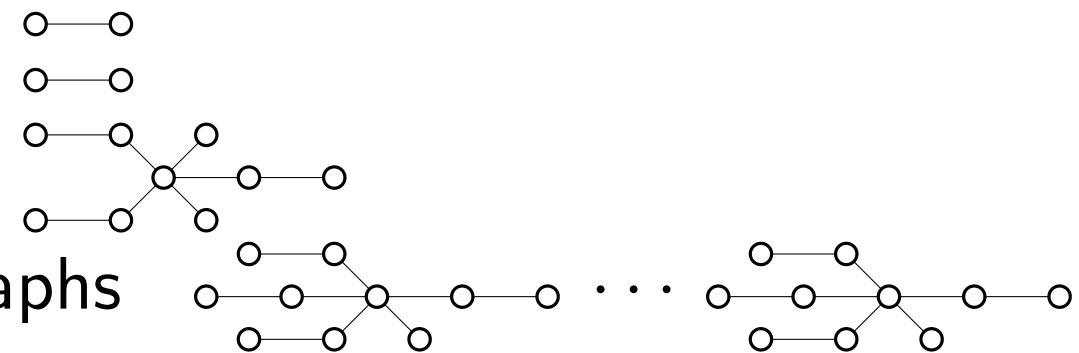
Hardness on Empty Hexasort:

- weak NP-hardness on two disjoint edges
- weak NP-hardness on a spider graph
- strong NP-hardness on disjoint spider graphs
- strong NP-hardness on trees



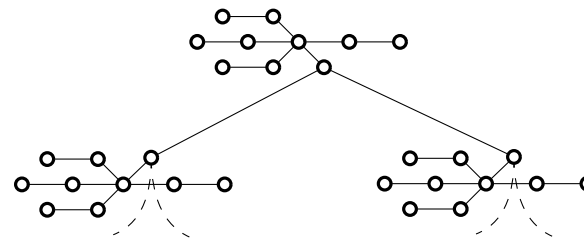
Hardness on Empty Hexasort:

- weak NP-hardness on two disjoint edges
- weak NP-hardness on a spider graph
- strong NP-hardness on disjoint spider graphs
- strong NP-hardness on trees



Algorithms on Hexasort:

- Dynamic Programm: $\mathcal{O}(|S| \cdot (|C| \cdot t)^{|V|})$
- Empty Hexasort in P with structural condition
- Fitting Hexasort is FPT: $\mathcal{O}(f(|C|, t) \cdot |S|^2)$



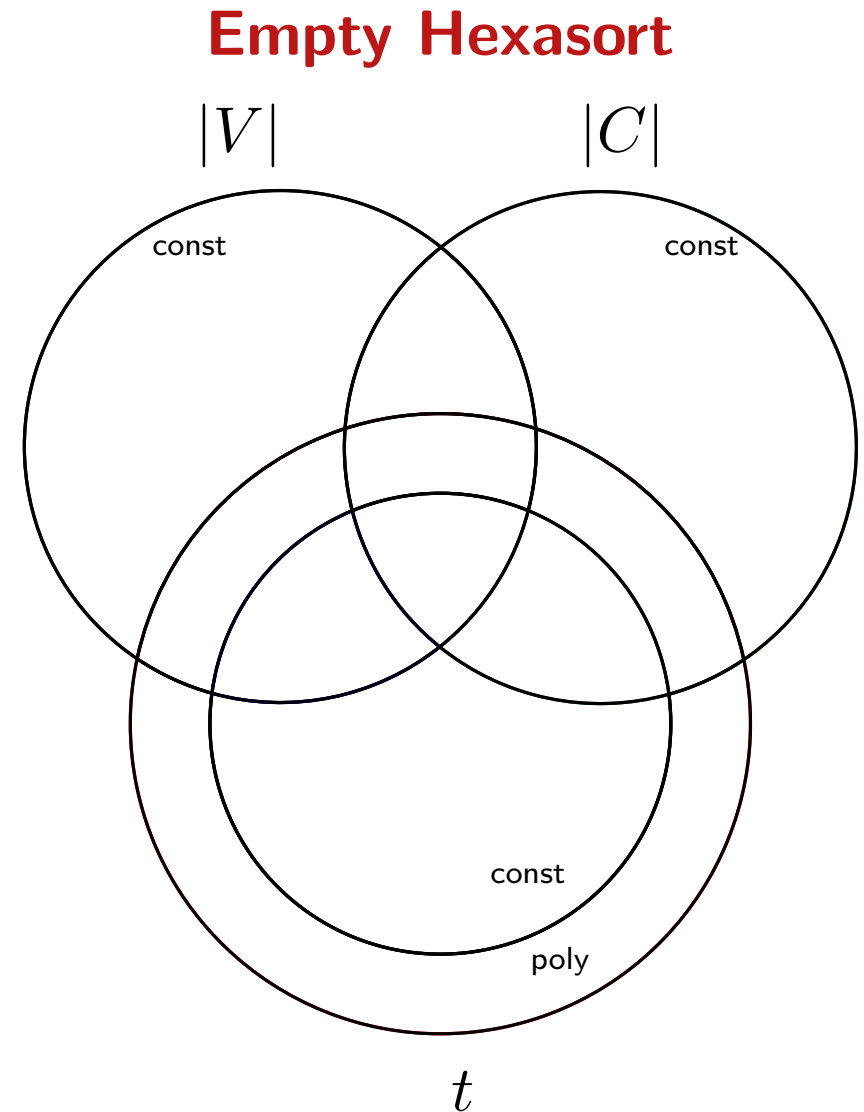
Hardness on Empty Hexasort:

- weak NP-hardness on two disjoint edges
- weak NP-hardness on a spider graph
- strong NP-hardness on disjoint spider graphs
- strong NP-hardness on trees

Algorithms on Hexasort:

- Dynamic Programm: $\mathcal{O}(|S| \cdot (|C| \cdot t)^{|V|})$
- Empty Hexasort in P with structural condition
- Fitting Hexasort is FPT: $\mathcal{O}(f(|C|, t) \cdot |S|^2)$

- Venn Diagram categorized by the parameters:
graph size $|V|$, number of colors $|C|$, and threshold t .



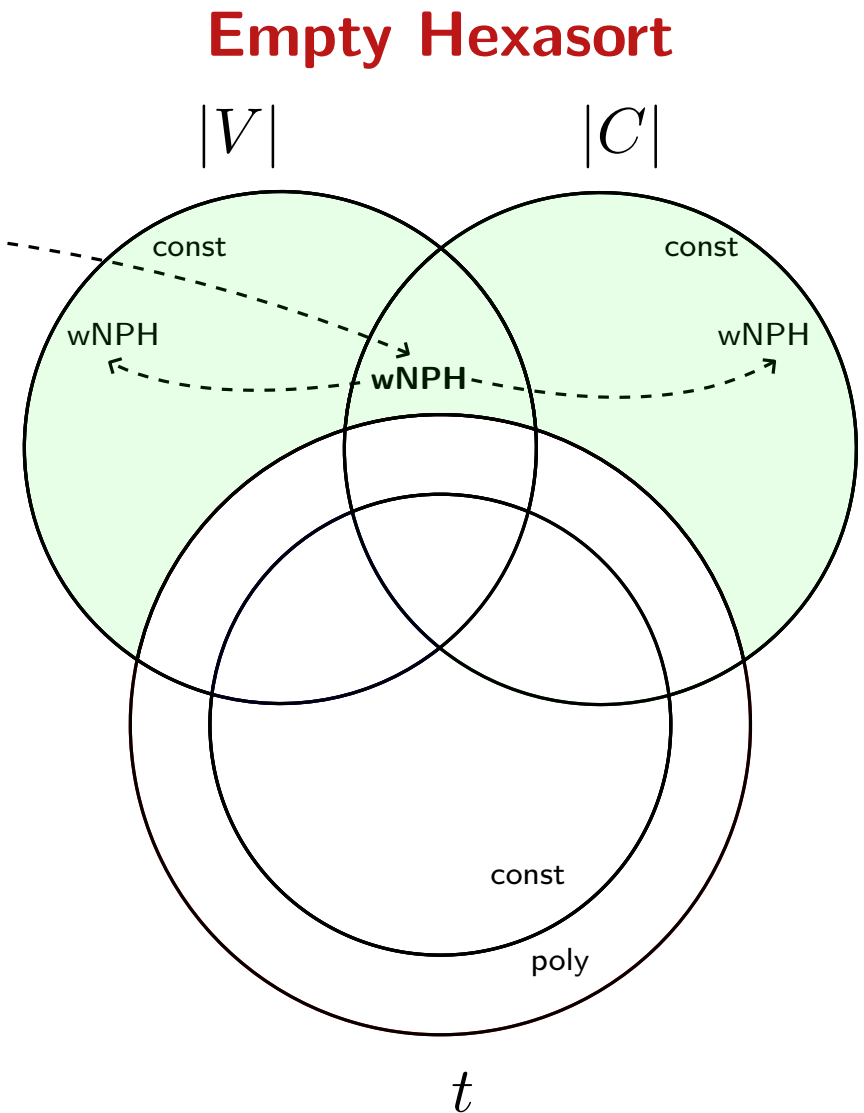
Hardness on Empty Hexasort:

- weak NP-hardness on two disjoint edges
- weak NP-hardness on a spider graph
- strong NP-hardness on disjoint spider graphs
- strong NP-hardness on trees

Algorithms on Hexasort:

- Dynamic Programm: $\mathcal{O}(|S| \cdot (|C| \cdot t)^{|V|})$
- Empty Hexasort in P with structural condition
- Fitting Hexasort is FPT: $\mathcal{O}(f(|C|, t) \cdot |S|^2)$

- Venn Diagram categorized by the parameters:
graph size $|V|$, number of colors $|C|$, and threshold t .



Hardness on Empty Hexasort:

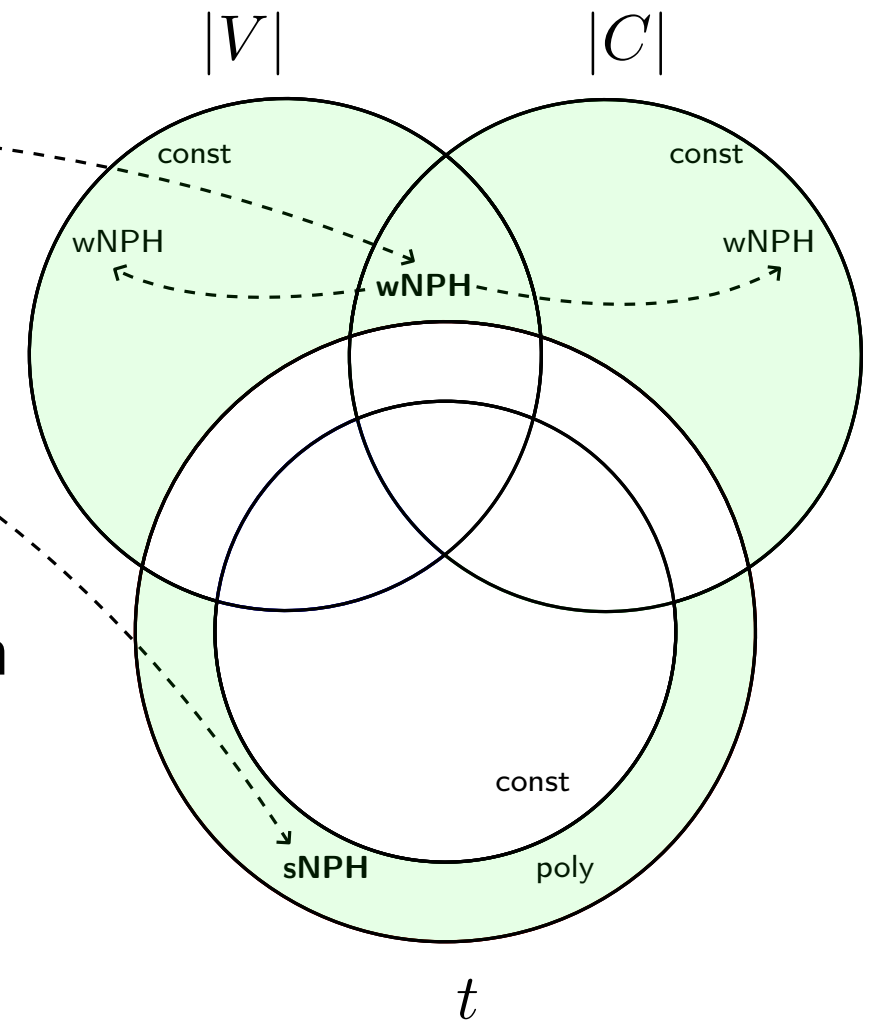
- weak NP-hardness on two disjoint edges
- weak NP-hardness on a spider graph
- strong NP-hardness on disjoint spider graphs
- strong NP-hardness on trees

Algorithms on Hexasort:

- Dynamic Programm: $\mathcal{O}(|S| \cdot (|C| \cdot t)^{|V|})$
- Empty Hexasort in P with structural condition
- Fitting Hexasort is FPT: $\mathcal{O}(f(|C|, t) \cdot |S|^2)$

- Venn Diagram categorized by the parameters:
graph size $|V|$, number of colors $|C|$, and threshold t .

Empty Hexasort



Hardness on Empty Hexasort:

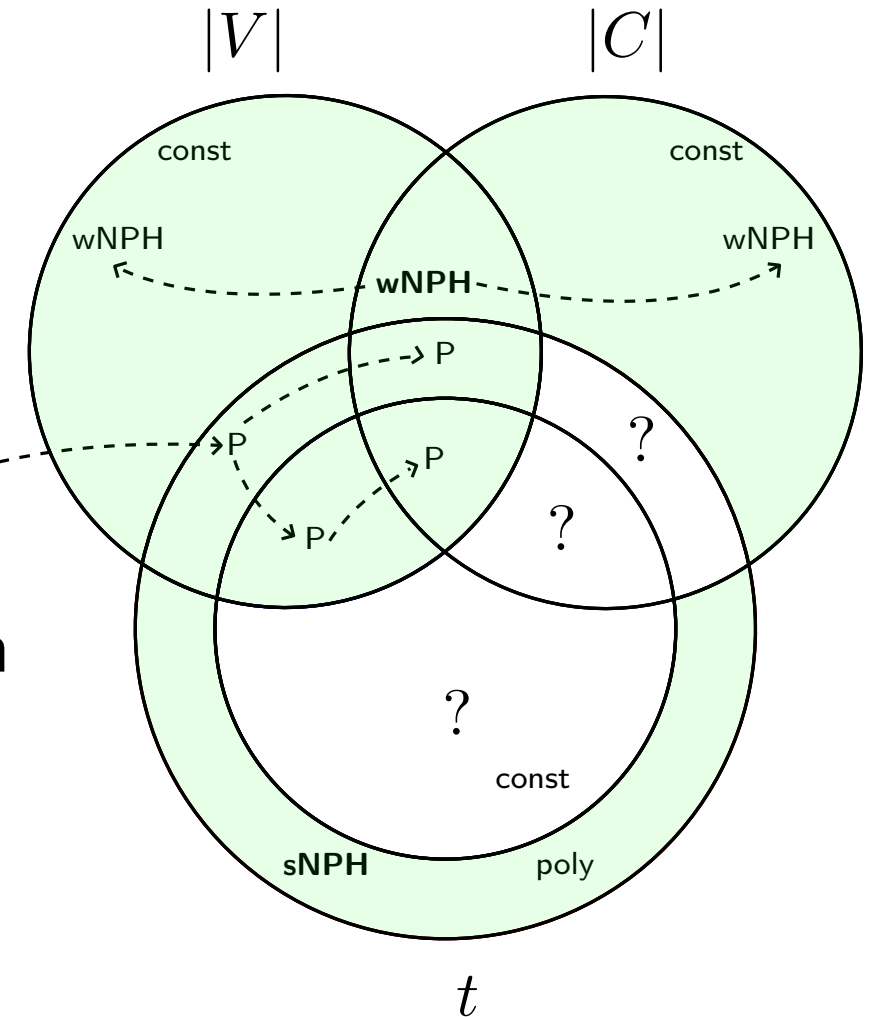
- weak NP-hardness on two disjoint edges
- weak NP-hardness on a spider graph
- strong NP-hardness on disjoint spider graphs
- strong NP-hardness on trees

Algorithms on Hexasort:

- Dynamic Programm: $\mathcal{O}(|S| \cdot (|C| \cdot t)^{|V|})$
- Empty Hexasort in P with structural condition
- Fitting Hexasort is FPT: $\mathcal{O}(f(|C|, t) \cdot |S|^2)$

- Venn Diagram categorized by the parameters:
graph size $|V|$, number of colors $|C|$, and threshold t .

Empty Hexasort



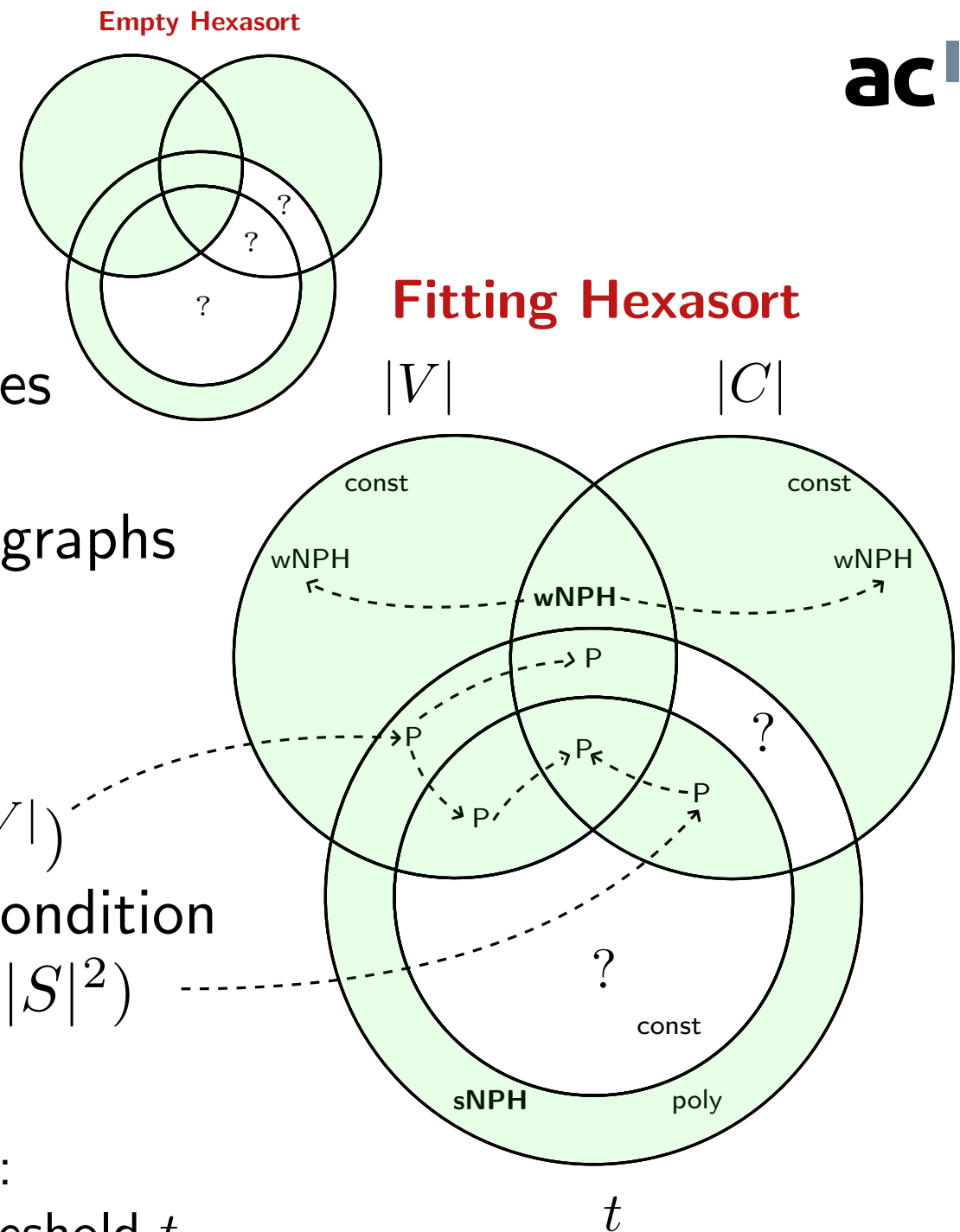
Hardness on Empty Hexasort:

- weak NP-hardness on two disjoint edges
- weak NP-hardness on a spider graph
- strong NP-hardness on disjoint spider graphs
- strong NP-hardness on trees

Algorithms on Hexasort:

- Dynamic Programm: $\mathcal{O}(|S| \cdot (|C| \cdot t)^{|V|})$
- Empty Hexasort in P with structural condition
- Fitting Hexasort is FPT: $\mathcal{O}(f(|C|, t) \cdot |S|^2)$

- Venn Diagram categorized by the parameters:
graph size $|V|$, number of colors $|C|$, and threshold t .

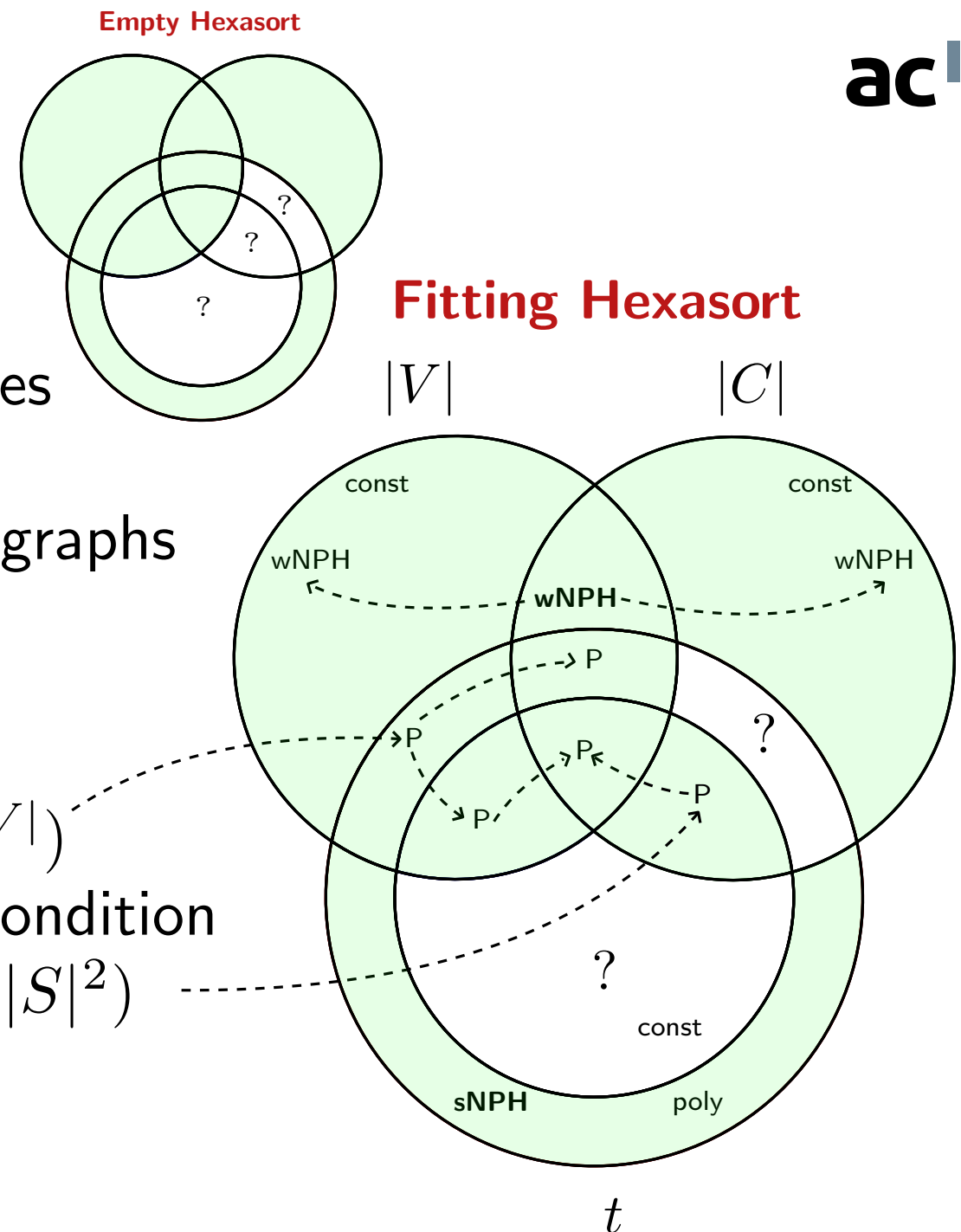


Hardness on Empty Hexasort:

- weak NP-hardness on two disjoint edges
- weak NP-hardness on a spider graph
- strong NP-hardness on disjoint spider graphs
- strong NP-hardness on trees

Algorithms on Hexasort:

- Dynamic Programm: $\mathcal{O}(|S| \cdot (|C| \cdot t)^{|V|})$
- Empty Hexasort in P with structural condition
- Fitting Hexasort is FPT: $\mathcal{O}(f(|C|, t) \cdot |S|^2)$
- weak NP-hardness is tight



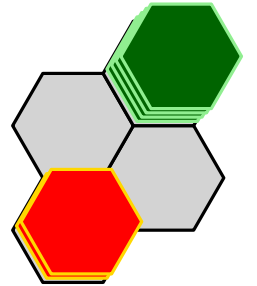
Weak NP-hardness via Partition

Partition

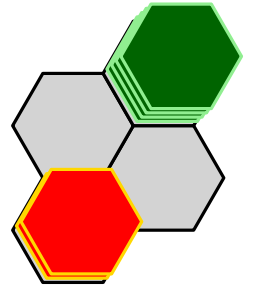
Input: A multiset P of positive integers $p_1, p_2, \dots, p_n$.

Task: Can P be split into two equal-sum parts?

■ is weakly NP-hard



Weak NP-hardness via Partition



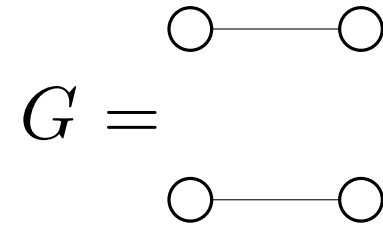
Partition

Input: A multiset P of positive integers $p_1, p_2, \dots, p_n$.

Task: Can P be split into two equal-sum parts?

■ is weakly NP-hard

Reduction:



$$P = \langle p_1, p_2, \dots, p_n \rangle \longrightarrow S = \langle (\text{black}, p_1), \dots, (\text{black}, p_n) \rangle$$

$$t = \sum P/2$$

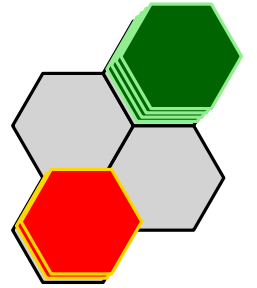
Weak NP-hardness via Partition

Partition

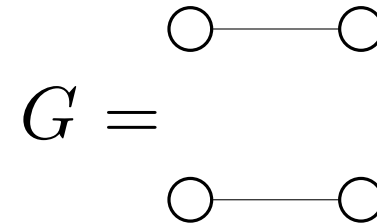
Input: A multiset P of positive integers $p_1, p_2, \dots, p_n$.

Task: Can P be split into two equal-sum parts?

■ is weakly NP-hard



Reduction:

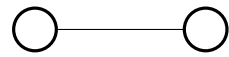
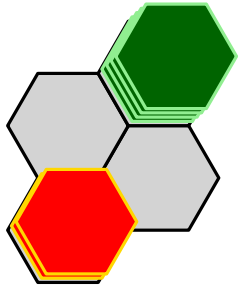


$$P = \langle p_1, p_2, \dots, p_n \rangle \longrightarrow S = \langle (\text{black}, p_1), \dots, (\text{black}, p_n) \rangle$$

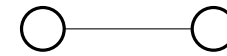
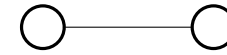
$$t = \sum P/2$$

Theorem 1: **Empty Hexasort** on two independent edges and a single color is weakly NP-hard.

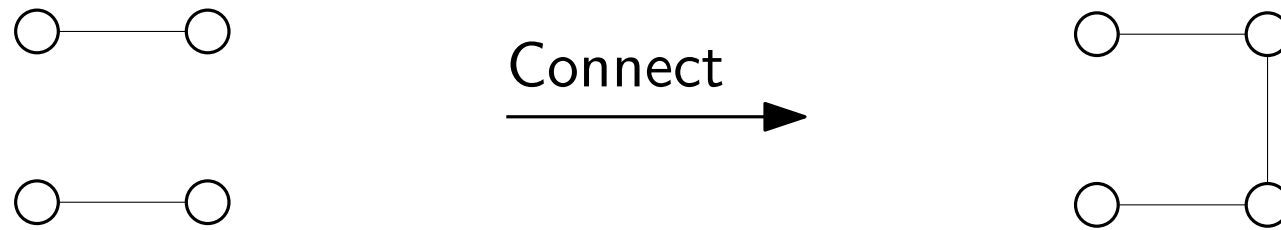
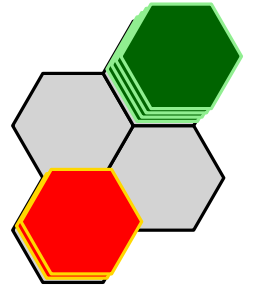
Weak NP-hardness on Connected Graphs



Connect
→

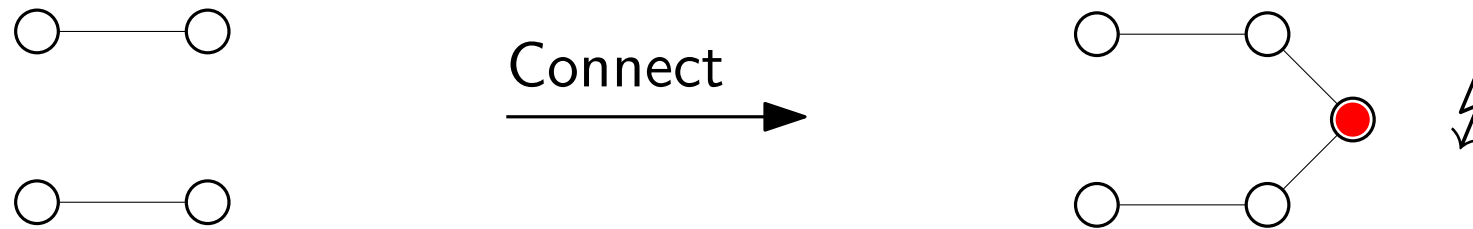
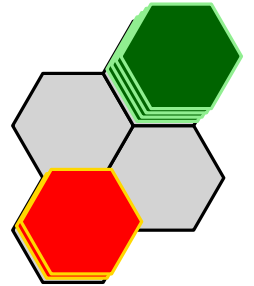


Weak NP-hardness on Connected Graphs



The two subsets could interact with each other!

Weak NP-hardness on Connected Graphs



Recall: We start with an empty board.

Idea: Force the placement of such a blocking vertex with a predetermined sequence.

Weak NP-hardness on Connected Graphs

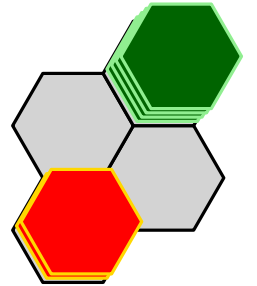
Forced three-merge:

$$G = \bigcirc - \bigcirc - \bigcirc - \bigcirc$$

$$S = \langle (\text{red}, 4), (\text{red}, 4), (\text{red}, 4) \rangle$$

$$t = 5$$

Empty Hexasort



Weak NP-hardness on Connected Graphs

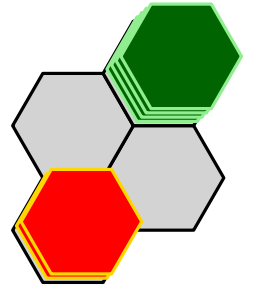
Forced three-merge:

$$G = \bigcirc - \bigcirc - \bigcirc - \bigcirc$$

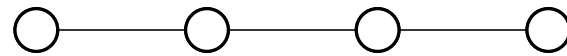
$$S = \langle (\text{red}, 4), (\text{red}, 4), (\text{red}, 4) \rangle$$

$$t = 5$$

Empty Hexasort



Example playthrough:



Weak NP-hardness on Connected Graphs

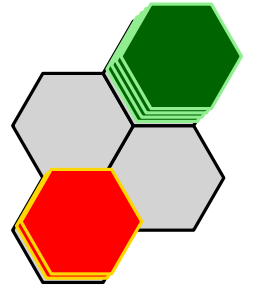
Forced three-merge:

$$G = \bigcirc - \bigcirc - \bigcirc - \bigcirc$$

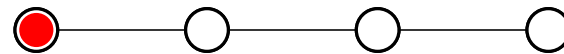
$$S = \langle (\text{red}, 4), (\text{red}, 4), (\text{red}, 4) \rangle$$

$$t = 5$$

Empty Hexasort



Example playthrough:



Weak NP-hardness on Connected Graphs

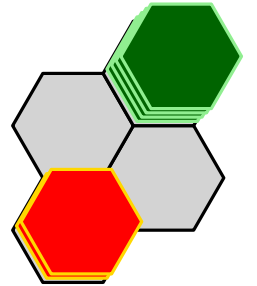
Forced three-merge:

$$G = \bigcirc - \bigcirc - \bigcirc - \bigcirc$$

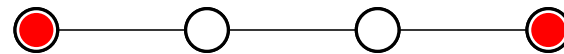
$$S = \langle (\text{red}, 4), (\text{red}, 4), (\text{red}, 4) \rangle$$

$$t = 5$$

Empty Hexasort



Example playthrough:



Weak NP-hardness on Connected Graphs

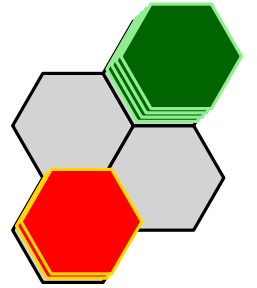
Forced three-merge:

$$G = \bigcirc - \bigcirc - \bigcirc - \bigcirc$$

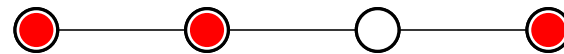
$$S = \langle (\text{red}, 4), (\text{red}, 4), (\text{red}, 4) \rangle$$

$$t = 5$$

Empty Hexasort



Example playthrough:



Weak NP-hardness on Connected Graphs

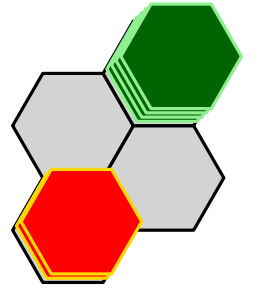
Forced three-merge:

$$G = \bigcirc - \bigcirc - \bigcirc - \bigcirc$$

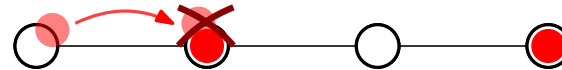
$$S = \langle (\text{red}, 4), (\text{red}, 4), (\text{red}, 4) \rangle$$

$$t = 5$$

Empty Hexasort



Example playthrough:



Weak NP-hardness on Connected Graphs

Forced three-merge:

$$G = \bigcirc - \bigcirc - \bigcirc - \bigcirc$$

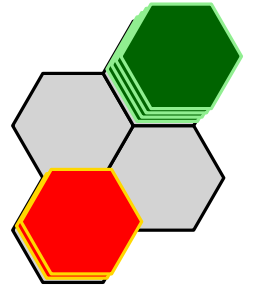
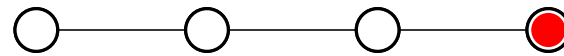
$$S = \langle (\text{red}, 4), (\text{red}, 4), (\text{red}, 4) \rangle$$

$$t = 5$$

Empty Hexasort



Example playthrough:



Weak NP-hardness on Connected Graphs

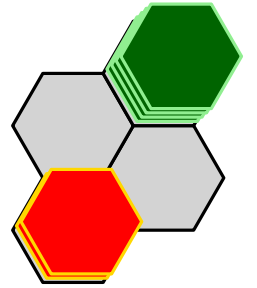
Forced three-merge:

$$G = \bigcirc - \bigcirc - \bigcirc - \bigcirc$$

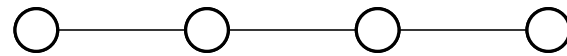
$$S = \langle (\text{red}, 4), (\text{red}, 4), (\text{red}, 4) \rangle$$

$$t = 5$$

Empty Hexasort



Example playthrough:



Weak NP-hardness on Connected Graphs

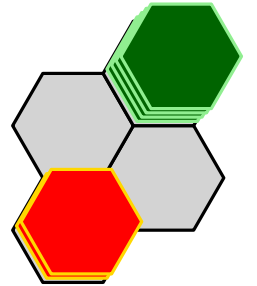
Forced three-merge:

$$G = \bigcirc - \bigcirc - \bigcirc - \bigcirc$$

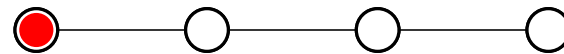
$$S = \langle (\text{red}, 4), (\text{red}, 4), (\text{red}, 4) \rangle$$

$$t = 5$$

Empty Hexasort



Example playthrough:



Weak NP-hardness on Connected Graphs

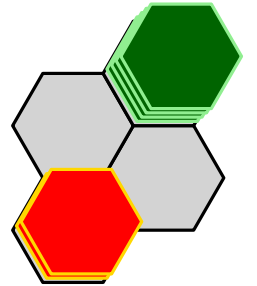
Forced three-merge:

$$G = \bigcirc - \bigcirc - \bigcirc - \bigcirc$$

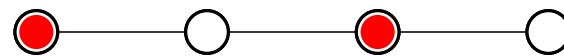
$$S = \langle (\text{red}, 4), (\text{red}, 4), (\text{red}, 4) \rangle$$

$$t = 5$$

Empty Hexasort



Example playthrough:



Weak NP-hardness on Connected Graphs

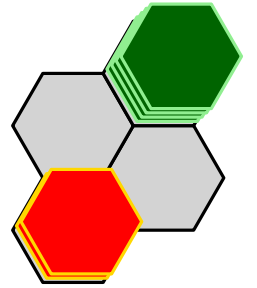
Forced three-merge:

$$G = \bigcirc - \bigcirc - \bigcirc - \bigcirc$$

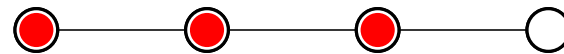
$$S = \langle (\text{red}, 4), (\text{red}, 4), (\text{red}, 4) \rangle$$

$$t = 5$$

Empty Hexasort



Example playthrough:



Weak NP-hardness on Connected Graphs

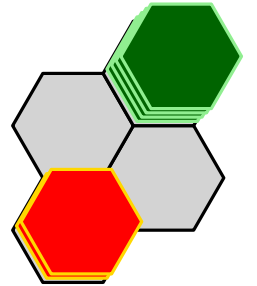
Forced three-merge:

$$G = \bigcirc - \bigcirc - \bigcirc - \bigcirc$$

$$S = \langle (\text{red}, 4), (\text{red}, 4), (\text{red}, 4) \rangle$$

$$t = 5$$

Empty Hexasort



Example playthrough:



The first two red stacks have to be placed exactly one vertex apart.

Weak NP-hardness on Connected Graphs

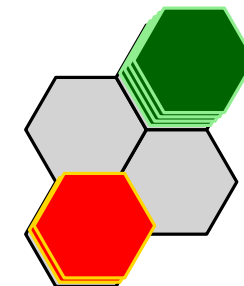
Forced three-merge:

$$G = \bigcirc - \bigcirc - \bigcirc - \bigcirc$$

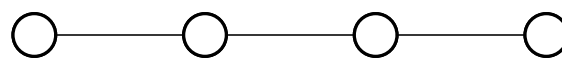
$$S = \langle (\text{red}, 4), (\text{red}, 4), (\text{red}, 4) \rangle$$

$$t = 5$$

Empty Hexasort ✓



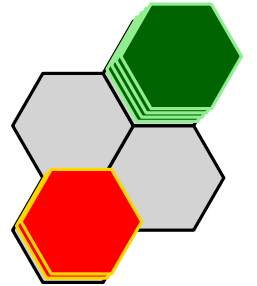
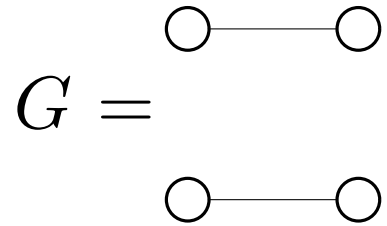
Example playthrough:



The first two red stacks have to be placed exactly one vertex apart.

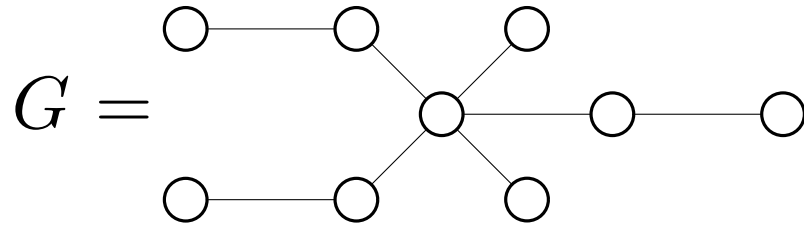
Weak NP-hardness on Connected Graphs

Reduction:

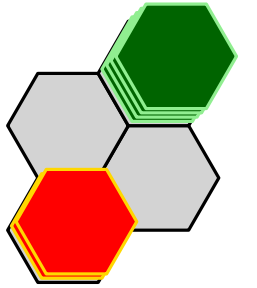


Weak NP-hardness on Connected Graphs

Reduction:

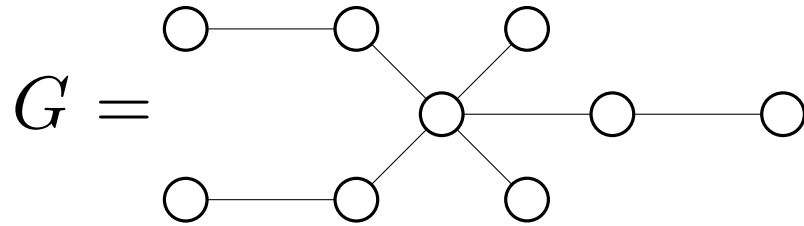


ac

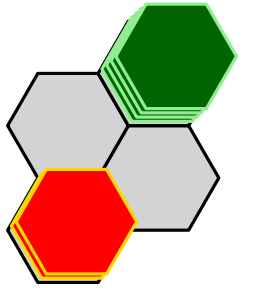


Weak NP-hardness on Connected Graphs

Reduction:

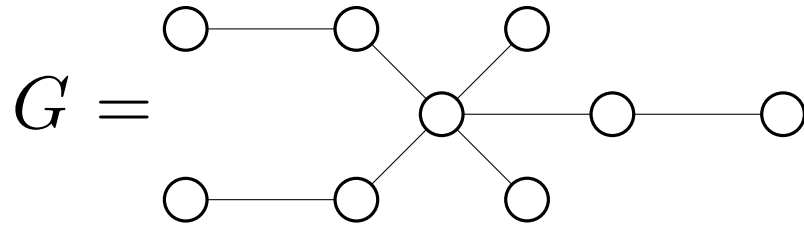


$$S' = \langle 2 \text{ red stacks}, 2 \text{ blue stacks}, 4 \text{ green stacks} \rangle + \\ S + \\ \langle 1 \text{ red stack}, 1 \text{ blue stack} \rangle$$



Weak NP-hardness on Connected Graphs

Reduction:

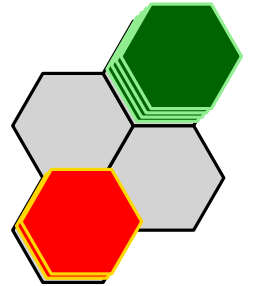


$$S' = \langle 2 \text{ red stacks}, 2 \text{ blue stacks}, 4 \text{ green stacks} \rangle +$$

$S +$

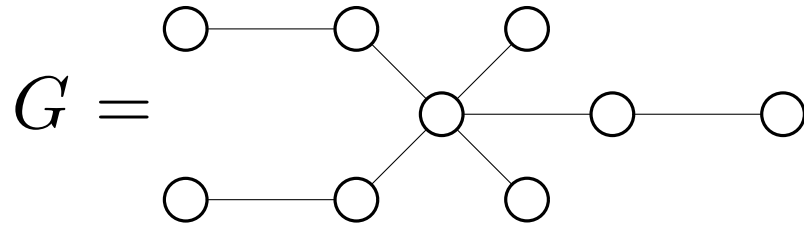
$$\langle 1 \text{ red stack}, 1 \text{ blue stack} \rangle$$

$$S = \langle (\text{black}, p_1), \dots, (\text{black}, p_n) \rangle$$

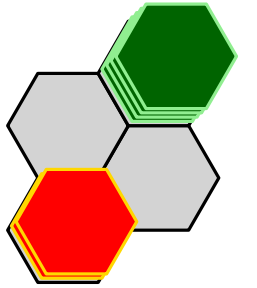


Weak NP-hardness on Connected Graphs

Reduction:

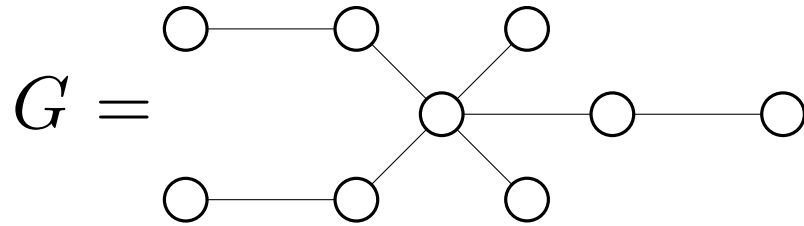


$$S' = \langle 2 \text{ red stacks}, 2 \text{ blue stacks}, 4 \text{ green stacks} \rangle + \\ S + \\ \langle 1 \text{ red stack}, 1 \text{ blue stack} \rangle$$



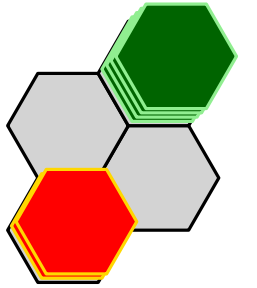
Weak NP-hardness on Connected Graphs

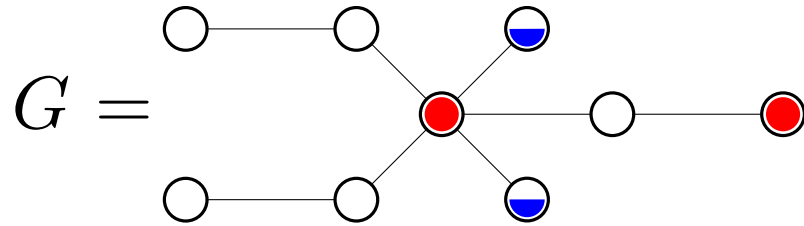
Reduction:



$$S' = \langle 2 \text{ red stacks}, 2 \text{ blue stacks}, 4 \text{ green stacks} \rangle + \\ S + \\ \langle 1 \text{ red stack}, 1 \text{ blue stack} \rangle$$

$$t = \sum P/2$$





$$S' = \langle 2 \text{ red stacks}, 2 \text{ blue stacks}, 4 \text{ green stacks} \rangle +$$

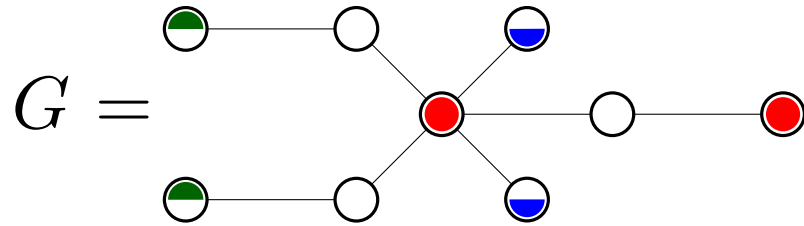
S_+

$$\langle 1 \text{ red stack}, 1 \text{ blue stack} \rangle$$

$$t = \sum P/2$$

Weak NP-hardness on Connected Graphs

Reduction:

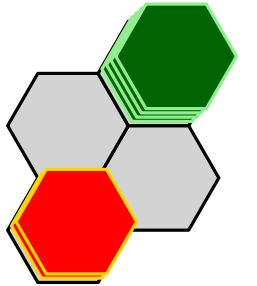


$$S' = \langle 2 \text{ red stacks}, 2 \text{ blue stacks}, 4 \text{ green stacks} \rangle +$$

$S +$

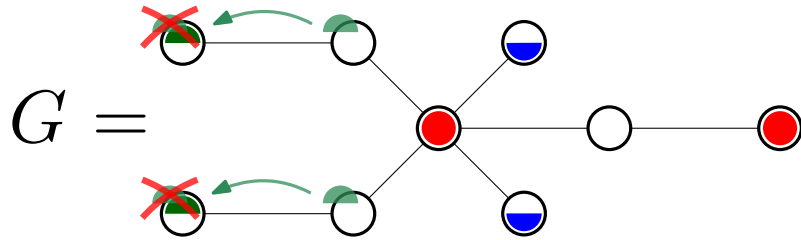
$$\langle 1 \text{ red stack}, 1 \text{ blue stack} \rangle$$

$$t = \sum P/2$$



Weak NP-hardness on Connected Graphs

Reduction:

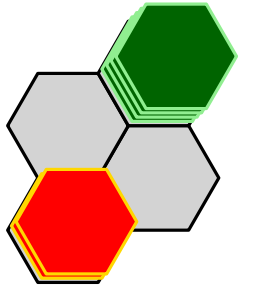


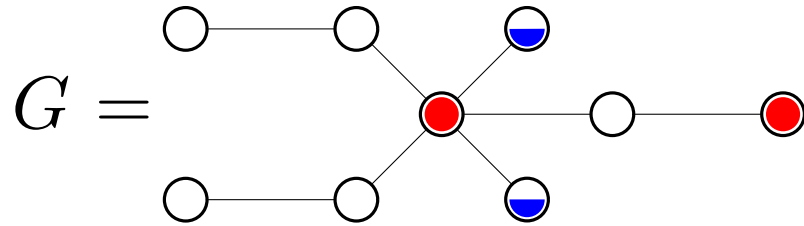
$$S' = \langle 2 \text{ red stacks}, 2 \text{ blue stacks}, 4 \text{ green stacks} \rangle +$$

$S +$

$$\langle 1 \text{ red stack}, 1 \text{ blue stack} \rangle$$

$$t = \sum P/2$$



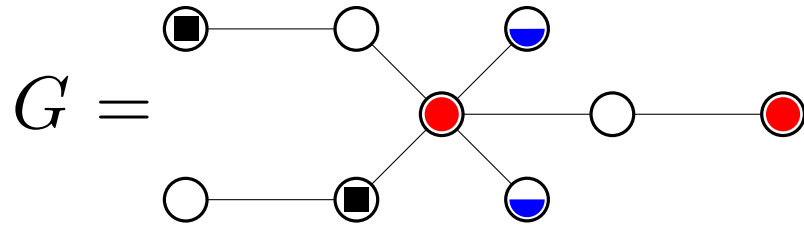


$$S' = \langle 2 \text{ red stacks}, 2 \text{ blue stacks}, 4 \text{ green stacks} \rangle + S + \langle 1 \text{ red stack}, 1 \text{ blue stack} \rangle$$

$$t = \sum P/2$$

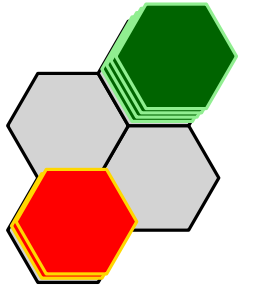
Weak NP-hardness on Connected Graphs

Reduction:



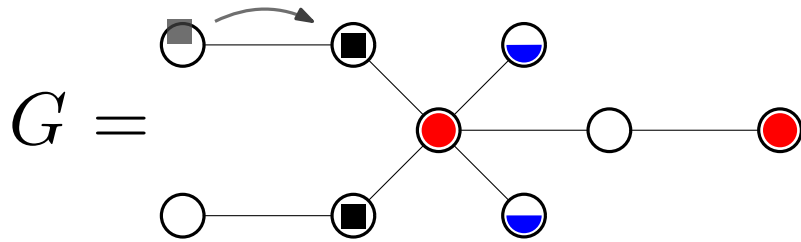
$$S' = \langle 2 \text{ red stacks}, 2 \text{ blue stacks}, 4 \text{ green stacks} \rangle +$$
$$\boxed{S} +$$
$$\langle 1 \text{ red stack}, 1 \text{ blue stack} \rangle$$

$$t = \sum P/2$$



Weak NP-hardness on Connected Graphs

Reduction:

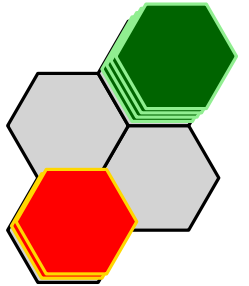


$$S' = \langle 2 \text{ red stacks}, 2 \text{ blue stacks}, 4 \text{ green stacks} \rangle +$$

$$S +$$

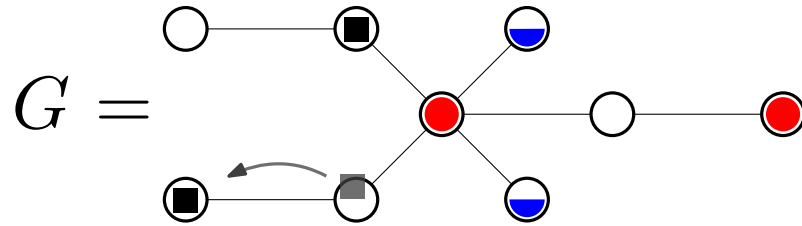
$$\langle 1 \text{ red stack}, 1 \text{ blue stack} \rangle$$

$$t = \sum P/2$$



Weak NP-hardness on Connected Graphs

Reduction:

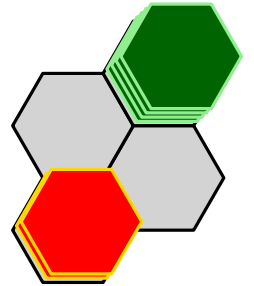


$$S' = \langle 2 \text{ red stacks}, 2 \text{ blue stacks}, 4 \text{ green stacks} \rangle +$$

$$S +$$

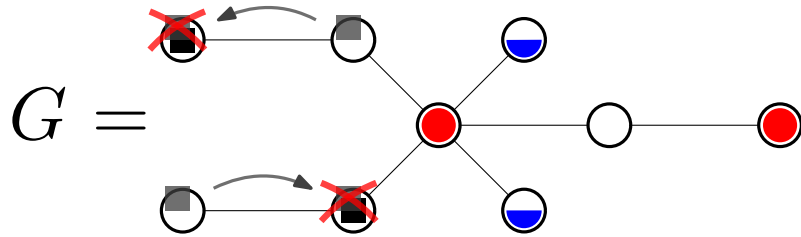
$$\langle 1 \text{ red stack}, 1 \text{ blue stack} \rangle$$

$$t = \sum P/2$$



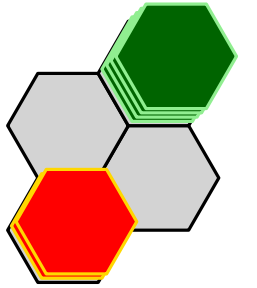
Weak NP-hardness on Connected Graphs

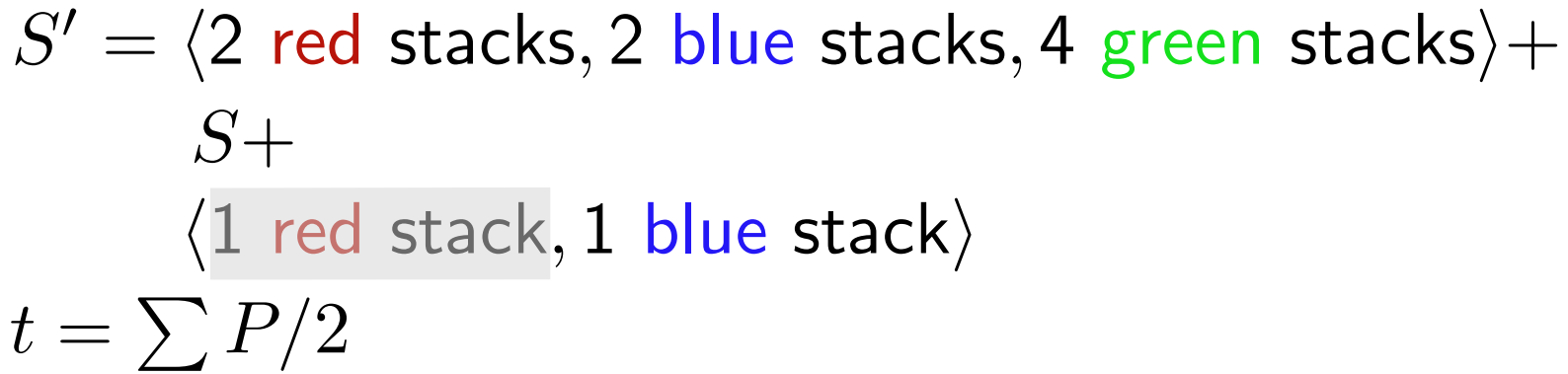
Reduction:



$$S' = \langle 2 \text{ red stacks}, 2 \text{ blue stacks}, 4 \text{ green stacks} \rangle +$$
$$S +$$
$$\langle 1 \text{ red stack}, 1 \text{ blue stack} \rangle$$

$$t = \sum P/2$$





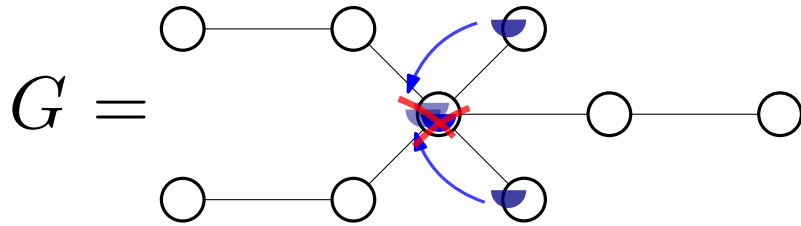
S_+

$$\langle 1 \text{ red stack}, 1 \text{ blue stack} \rangle$$

$$t = \sum P/2$$

Weak NP-hardness on Connected Graphs

Reduction:

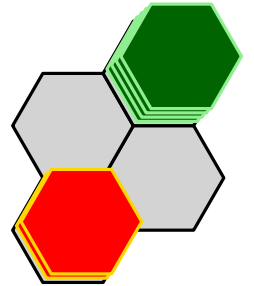


$$S' = \langle 2 \text{ red stacks}, 2 \text{ blue stacks}, 4 \text{ green stacks} \rangle +$$

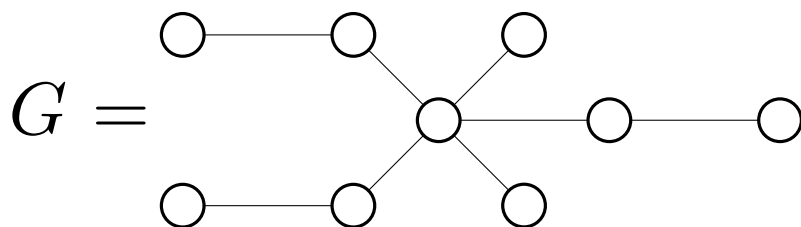
$S +$

$$\langle 1 \text{ red stack}, 1 \text{ blue stack} \rangle$$

$$t = \sum P/2$$



Reduction:



$$S' = \langle 2 \text{ red st}, 2 \text{ blue st}, 4 \text{ green st} \rangle +$$

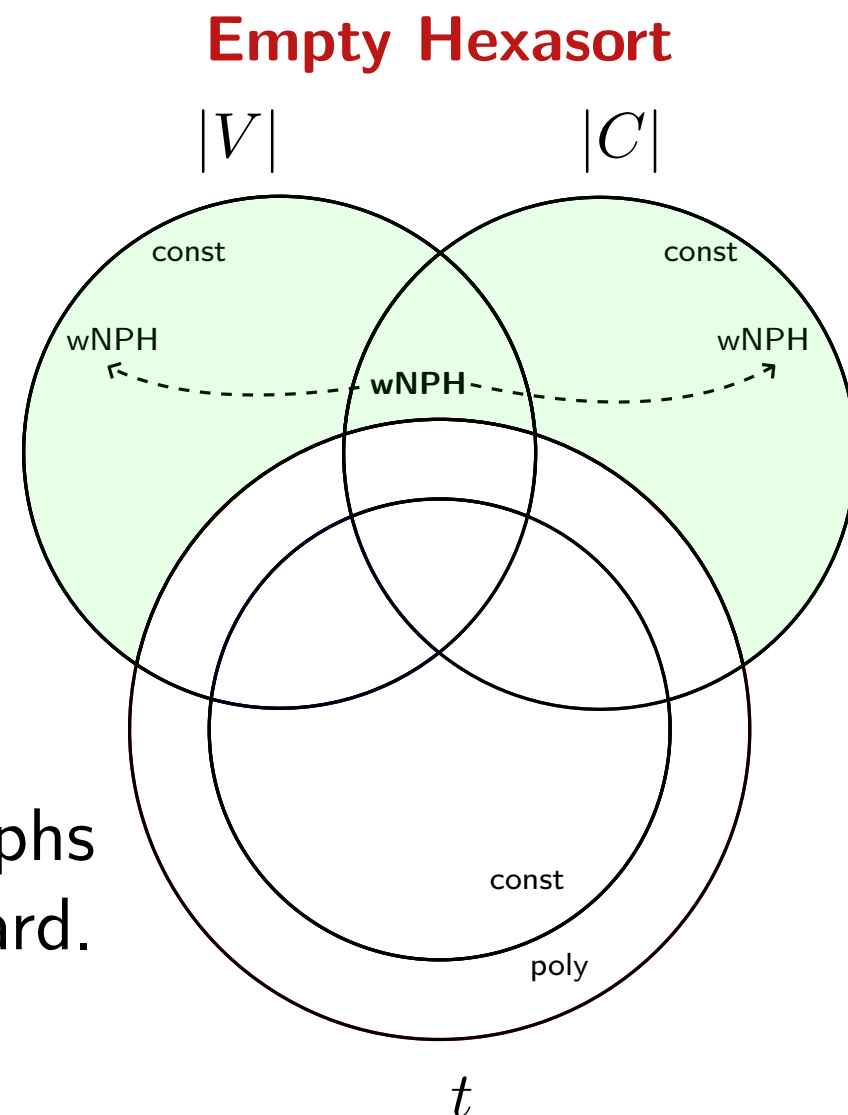
$$S +$$

$$\langle 1 \text{ red stack}, 1 \text{ blue stack} \rangle$$

$$t = \sum P/2$$

Theorem 2. **Empty Hexasort** on spider graphs with nine vertices and four colors is weakly NP-hard.

Recall: Empty $\leq$ Fitting



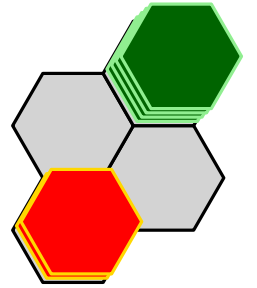
Strong NP-hardness via 3-Partition

3-Partition

Input: A multiset A of $3m$ positive integers and a bound $B \in \mathbb{Z}^+$.

Task: Can A be partitioned into m triplets $A_1, A_2, \dots, A_m$ such that each triplet sums up to exactly B .

■ is strongly NP-hard

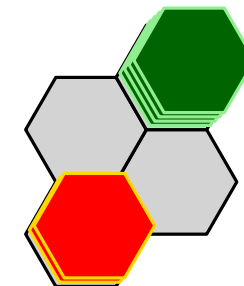


Strong NP-hardness via 3-Partition

3-Partition

Input: A multiset A of $3m$ positive integers and a bound $B \in \mathbb{Z}^+$.

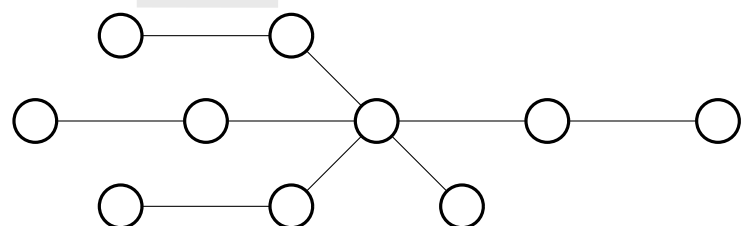
Task: Can A be partitioned into m triplets $A_1, A_2, \dots, A_m$ such that each triplet sums up to exactly B .



■ is strongly NP-hard

Reduction:

$$G = m \times$$



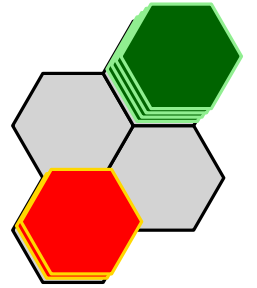
$$S = S_A + \langle 2 \text{ red stacks}, 2 \text{ blue stacks}, 2 \text{ green stacks} \rangle \times m + \quad t = B + 4$$
$$\langle 1 \text{ red stack}, 1 \text{ blue stack}, 1 \text{ green stack}, 4 \text{ black stacks} \rangle \times m$$

Strong NP-hardness via 3-Partition

3-Partition

Input: A multiset A of $3m$ positive integers and a bound $B \in \mathbb{Z}^+$.

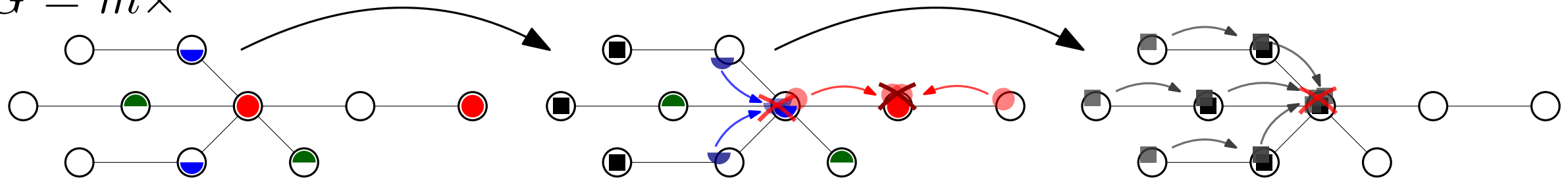
Task: Can A be partitioned into m triplets $A_1, A_2, \dots, A_m$ such that each triplet sums up to exactly B .



■ is strongly NP-hard

Reduction:

$G = m \times$



$\langle 2 \text{ red stacks}, 2 \text{ blue stacks}, 2 \text{ green stacks} \rangle \times m + t = B + 4$

$S = S_A +$

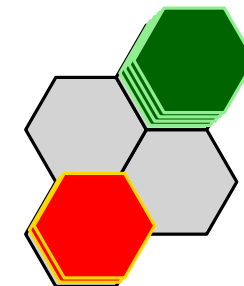
$\langle 1 \text{ red stack}, 1 \text{ blue stack}, 1 \text{ green stack}, 4 \text{ black stacks} \rangle \times m$

Strong NP-hardness via 3-Partition

3-Partition

Input: A multiset A of $3m$ positive integers and a bound $B \in \mathbb{Z}^+$.

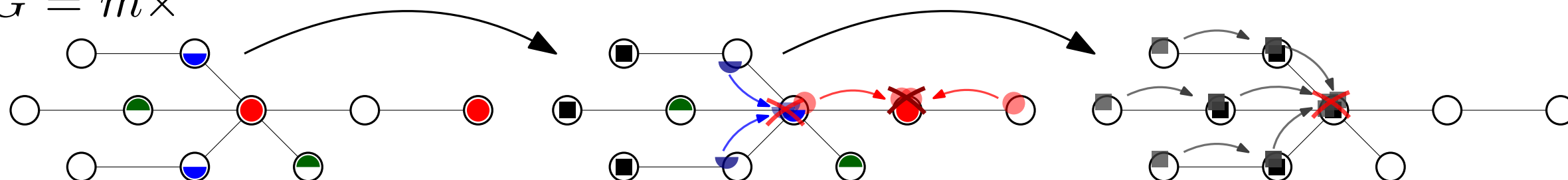
Task: Can A be partitioned into m triplets $A_1, A_2, \dots, A_m$ such that each triplet sums up to exactly B .



■ is strongly NP-hard

Reduction:

$G = m \times$



$\langle 2 \text{ red stacks, } 2 \text{ blue stacks, } 2 \text{ green stacks} \rangle \times m + t = B + 4$

$S = S_A +$

$\langle 1 \text{ red stack, } 1 \text{ blue stack, } 1 \text{ green stack, } 4 \text{ black stacks} \rangle \times m$

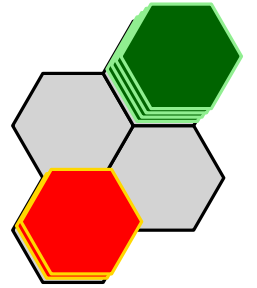
By the pigeon-hole principle, each gadget must be assigned the colorful stacks shown in the first configuration.

Strong NP-hardness via 3-Partition

3-Partition

Input: A multiset A of $3m$ positive integers and a bound $B \in \mathbb{Z}^+$.

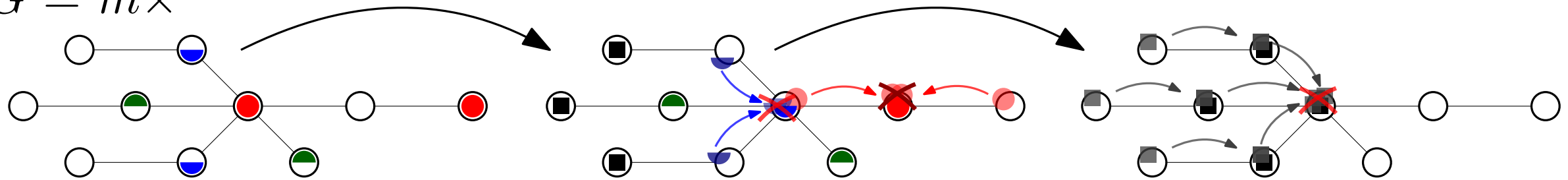
Task: Can A be partitioned into m triplets $A_1, A_2, \dots, A_m$ such that each triplet sums up to exactly B .



■ is strongly NP-hard

Reduction:

$G = m \times$



$\langle 2 \text{ red stacks}, 2 \text{ blue stacks}, 2 \text{ green stacks} \rangle \times m + t = B + 4$

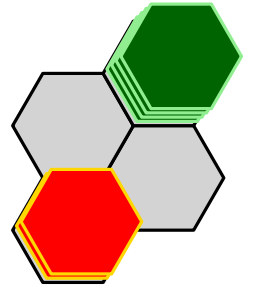
$S = S_A +$

$\langle 1 \text{ red stack}, 1 \text{ blue stack}, 1 \text{ green stack}, 4 \text{ black stacks} \rangle \times m$

Theorem 3. Empty Hexasort on the disjoint union of spider graphs with ten vertices each is strongly NP-hard.

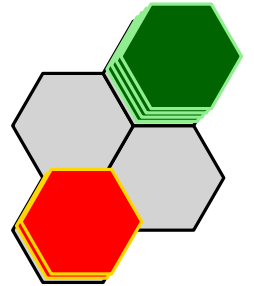
Strong NP-hardness on Connected Graphs

- Is based on the previous reduction from 3-partition
- The sequence and threshold stay the same; only the graph structure changes.
- The connectors do not create new useful interactions.

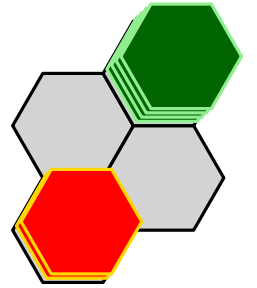


Strong NP-hardness on Connected Graphs

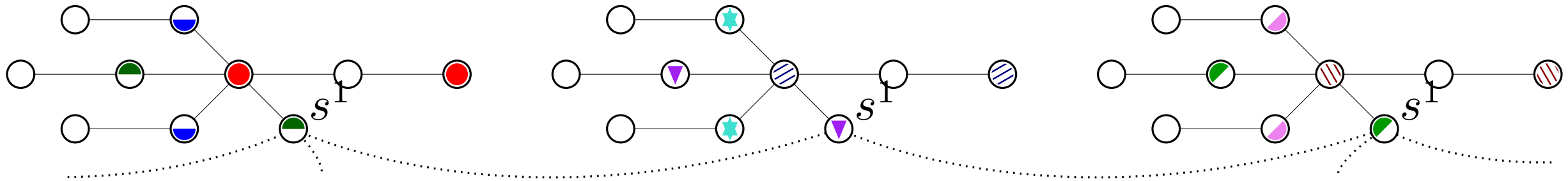
- Is based on the previous reduction from 3-partition
- The sequence and threshold stay the same; only the graph structure changes.
- The connectors do not create new useful interactions.



Strong NP-hardness on Connected Graphs

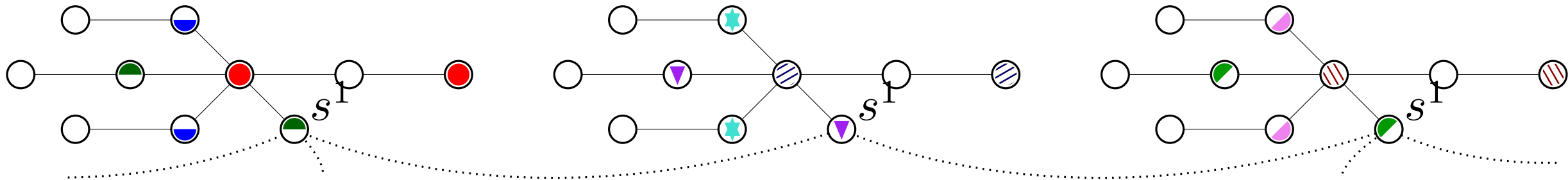


- Is based on the previous reduction from 3-partition
- The sequence and threshold stay the same; only the graph structure changes.
- The connectors do not create new useful interactions.

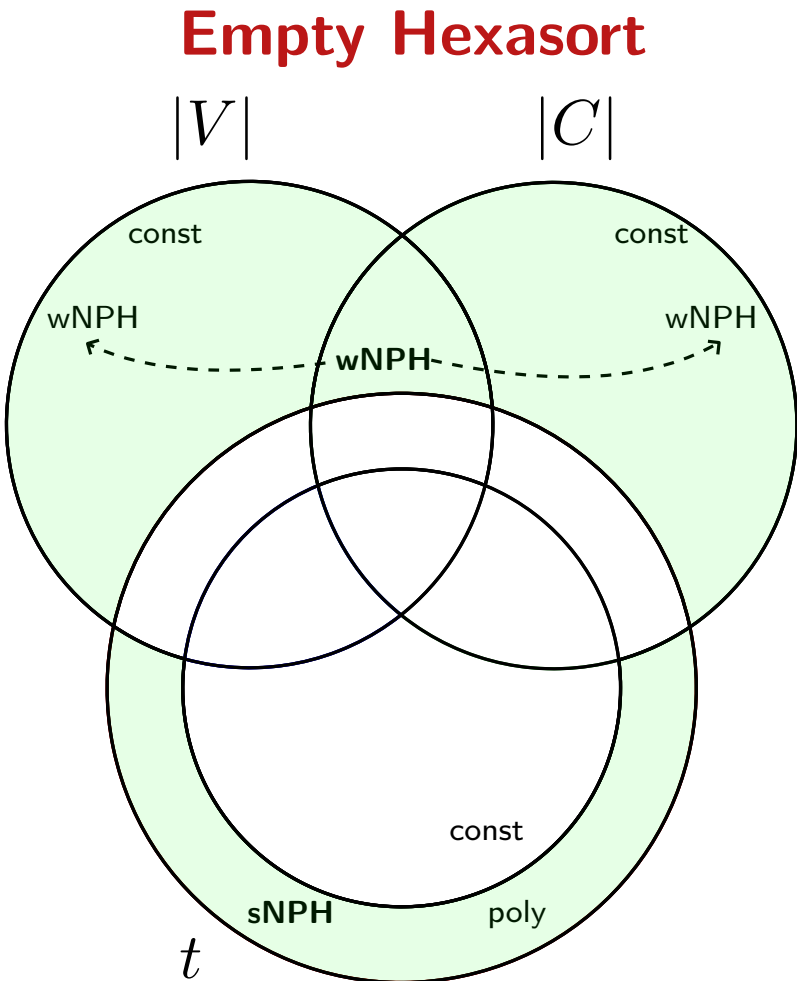


Theorem 4. **Empty Hexasort** on trees of bounded height or degree is strongly NP-hard.

Strong NP-hardness on Connected Graphs



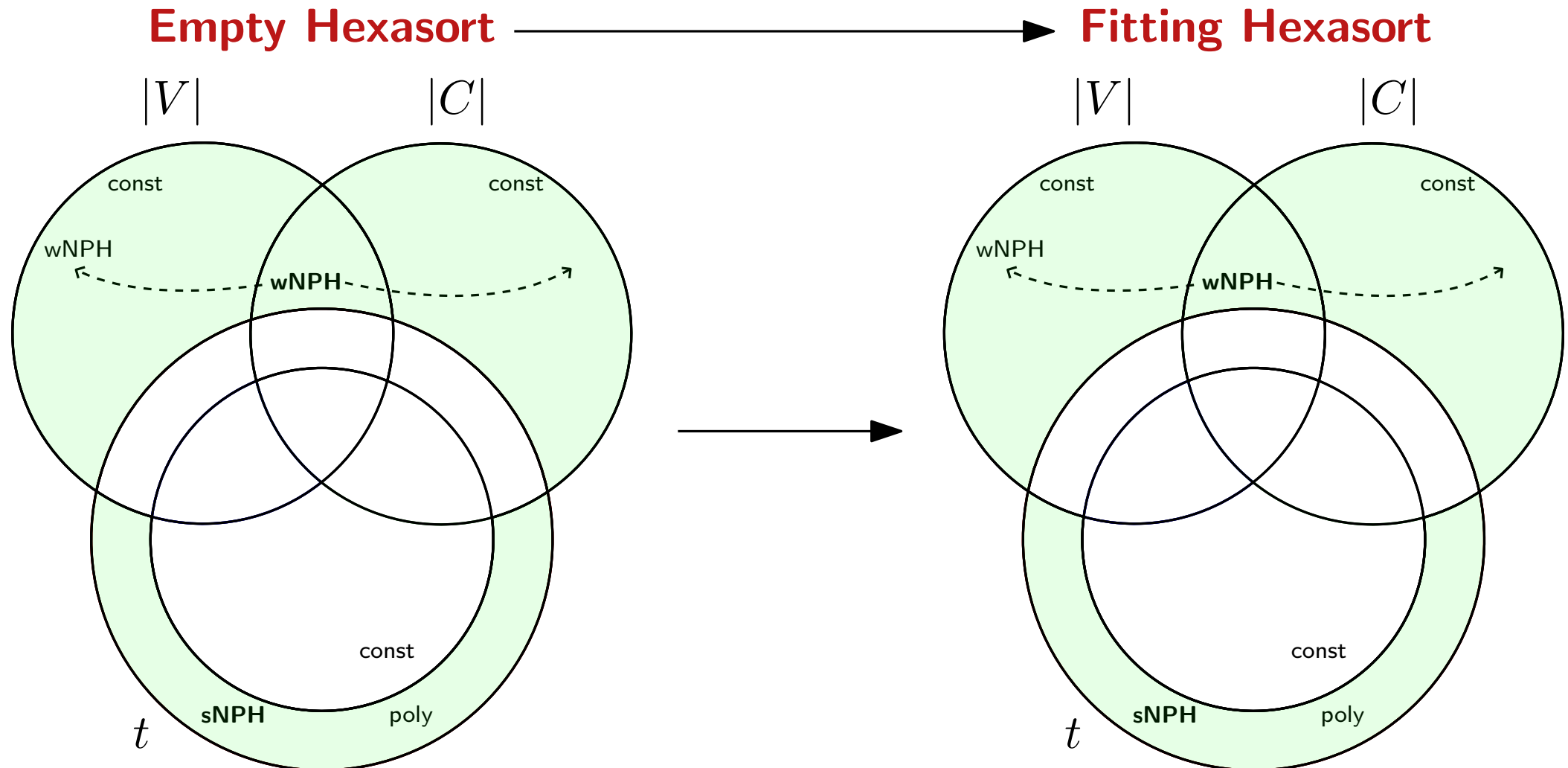
Theorem 4. **Empty Hexasort** on trees of bounded height or degree is strongly NP-hard.



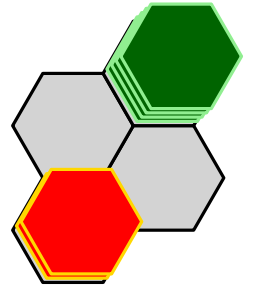
NP-hardness summary

Theorem 2. **Empty Hexasort** on spider graphs with nine vertices and four colors is weakly NP-hard.

Theorem 4. **Empty Hexasort** on trees of bounded height or degree is strongly NP-hard.

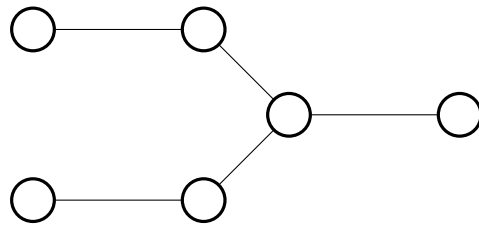


Theorem 5. **Fitting** and **Empty Hexasort** can be decided in time $\mathcal{O}(|S| \cdot (|C| \cdot t)^{|V|})$.



This dynamic program explores the space of reachable board configurations.

Lemma 6. **Empty Hexasort** is positive if G contains number of colors $|C|$ non-overlapping specific spider graphs.

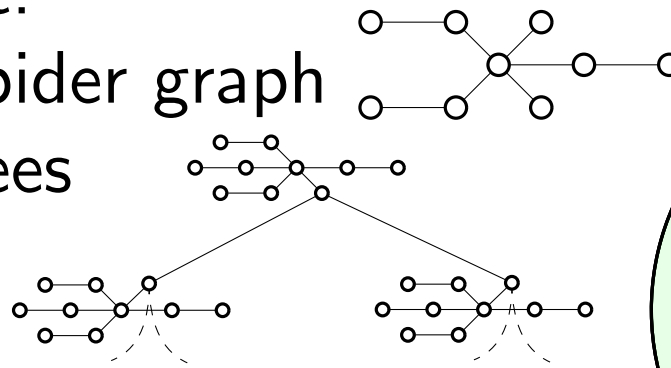


Theorem 7. **Fitting Hexasort** can be decided in time $\mathcal{O}(f(|C|, t) \cdot |S|^2)$ for some function f .

By handling P-time solvable instances first, we restrict the analysis to large graphs with bounded degree and small vertex cover.

Hardness on Empty Hexasort:

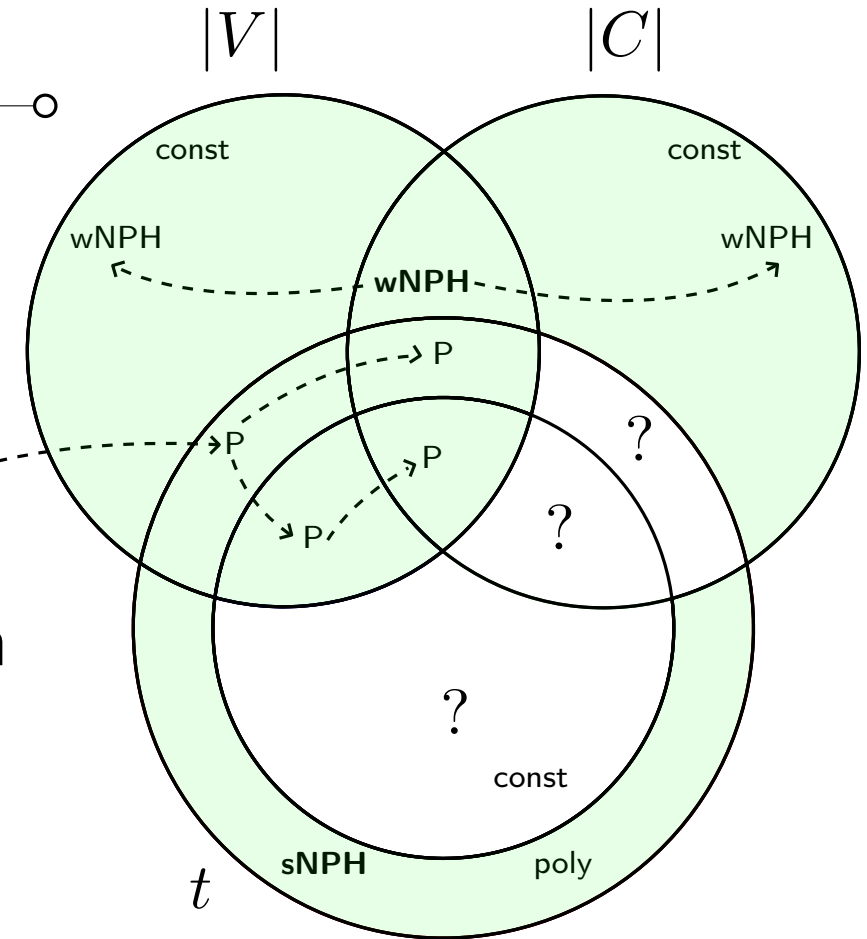
- weak NP-hardness on a spider graph
- strong NP-hardness on trees



Algorithms on Hexasort:

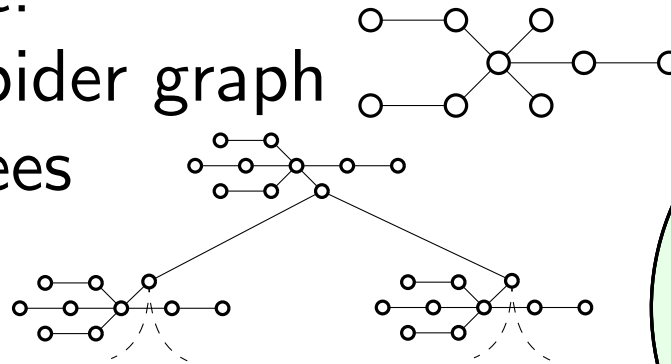
- Dynamic Programm: $\mathcal{O}(|S| \cdot (|C| \cdot t)^{|V|})$
- Empty Hexasort in P with structural condition
- Fitting Hexasort is FPT: $\mathcal{O}(f(|C|, t) \cdot |S|^2)$

Empty Hexasort



Hardness on Empty Hexasort:

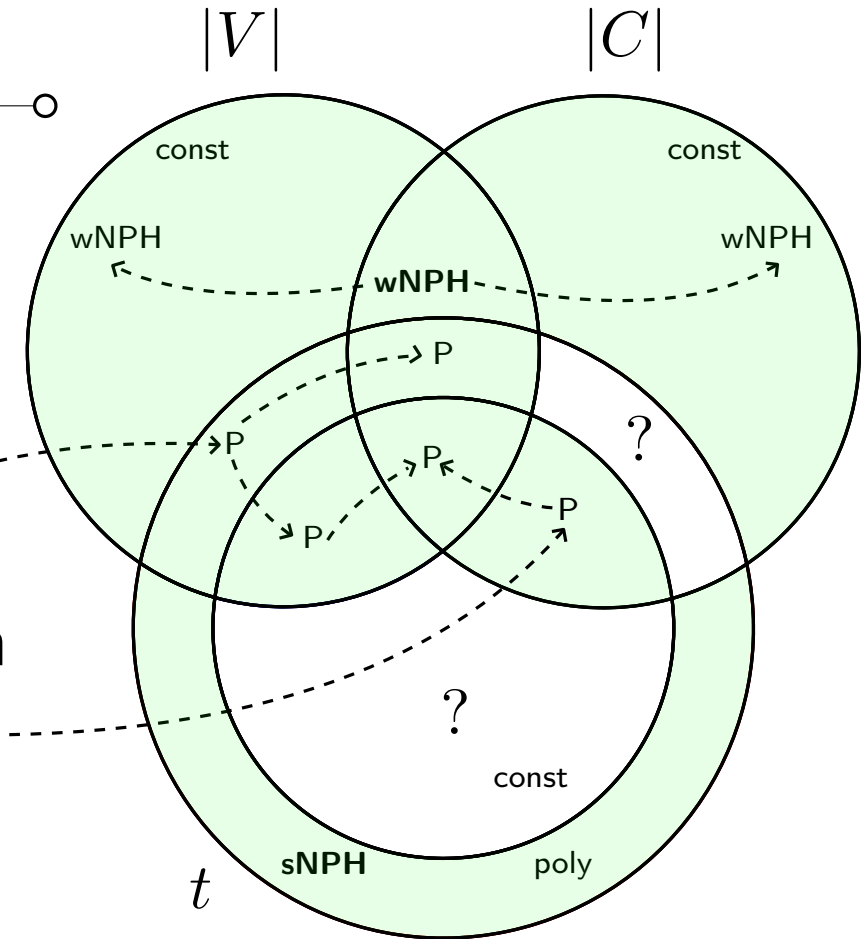
- weak NP-hardness on a spider graph
- strong NP-hardness on trees



Algorithms on Hexasort:

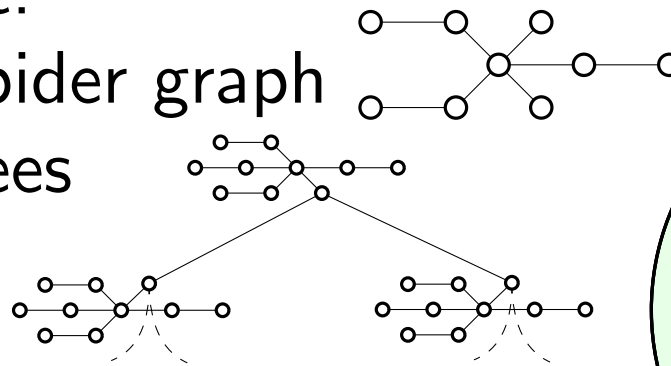
- Dynamic Programm: $\mathcal{O}(|S| \cdot (|C| \cdot t)^{|V|})$
- Empty Hexasort in P with structural condition
- Fitting Hexasort is FPT: $\mathcal{O}(f(|C|, t) \cdot |S|^2)$

Fitting Hexasort



Hardness on Empty Hexasort:

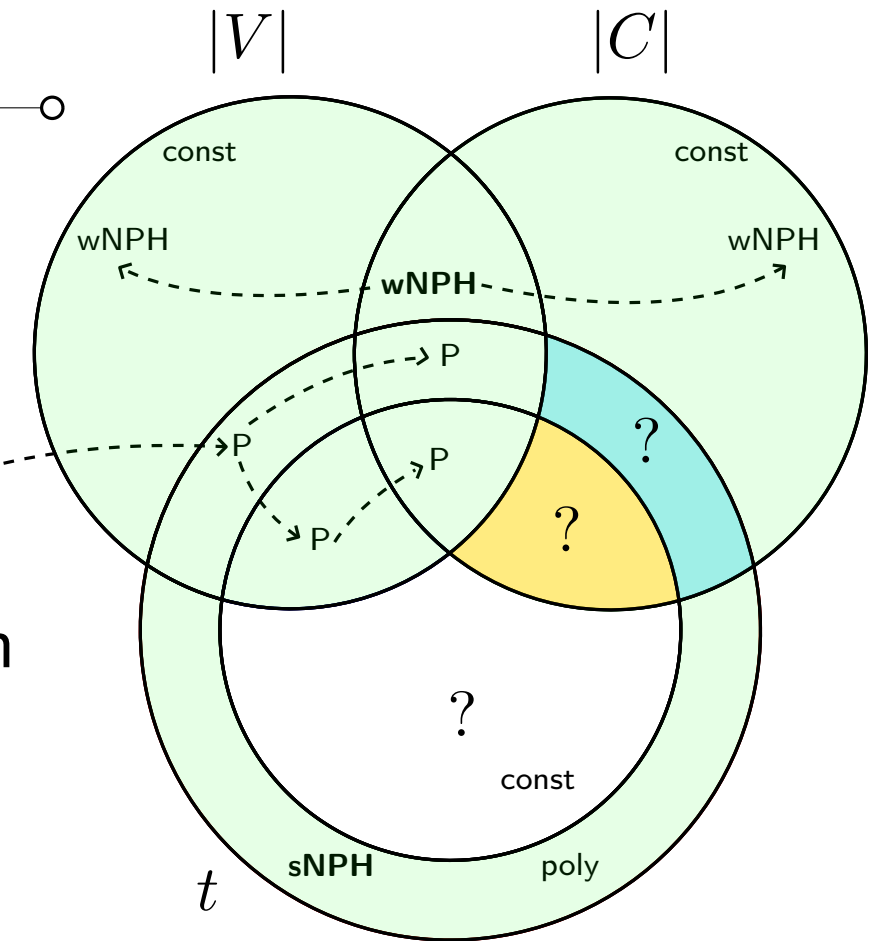
- weak NP-hardness on a spider graph
- strong NP-hardness on trees



Algorithms on Hexasort:

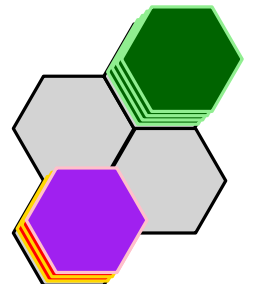
- Dynamic Programm: $\mathcal{O}(|S| \cdot (|C| \cdot t)^{|V|})$
- Empty Hexasort in P with structural condition
- Fitting Hexasort is FPT: $\mathcal{O}(f(|C|, t) \cdot |S|^2)$

Empty Hexasort



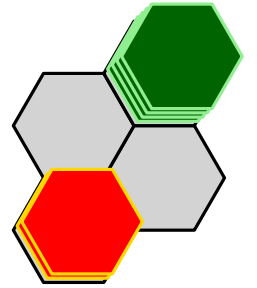
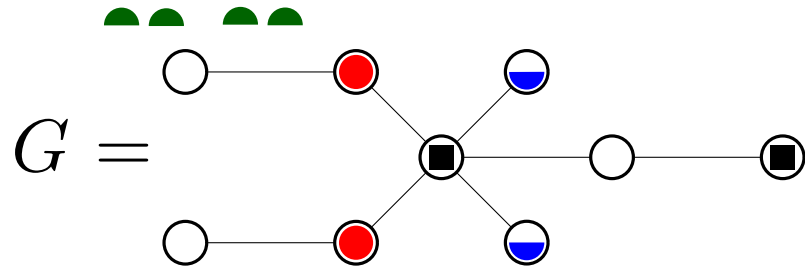
Open directions:

- Empty Hexasort tractability for bounded t and bounded $|C|$
- Strong NP-hardness for constant $|C|$
- Multicolored stacks as in playable versions



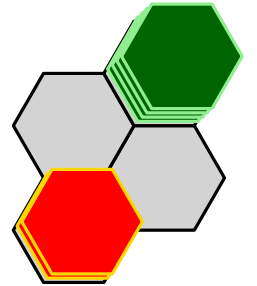
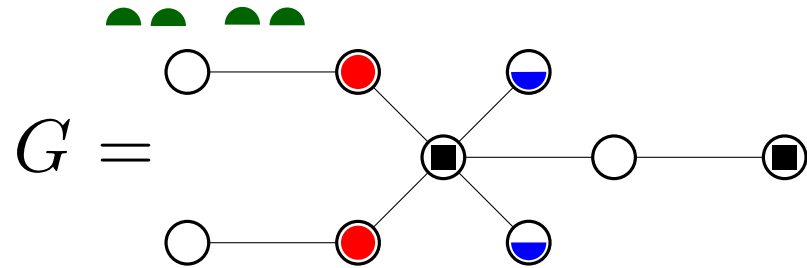
Possible Questions and Answers

- Why do we need the **green** stacks in the reduction from Partition to Empty Hexasort on a connected graph?

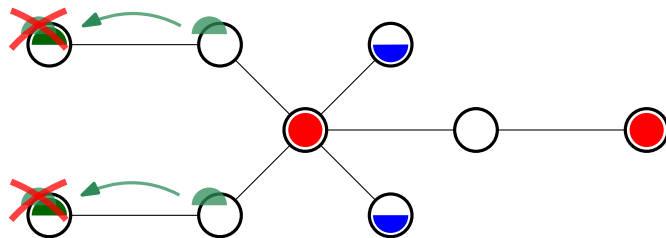


Possible Questions and Answers

- Why do we need the **green** stacks in the reduction from Partition to Empty Hexasort on a connected graph?

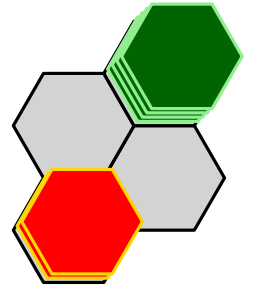
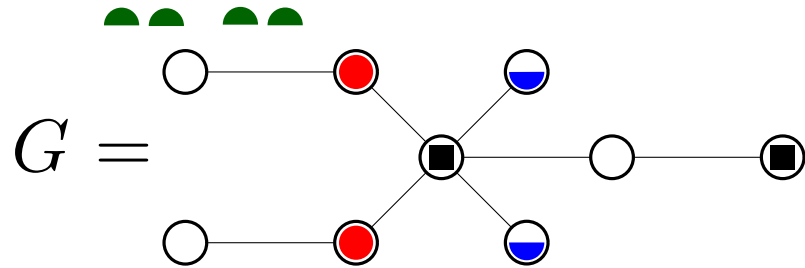


Because otherwise the placement is not forced to reserve two edges for the black partition stacks.

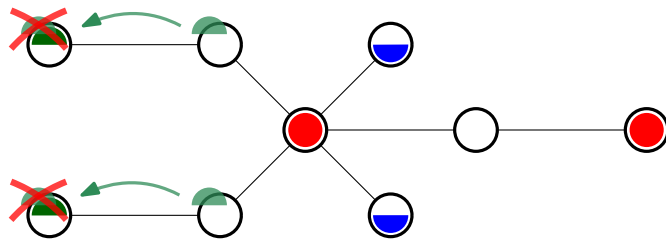


Possible Questions and Answers

- Why do we need the **green** stacks in the reduction from Partition to Empty Hexasort on a connected graph?

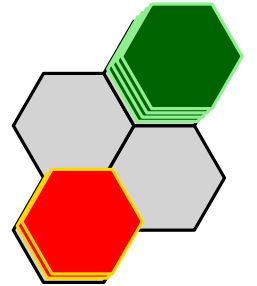


Because otherwise the placement is not forced to reserve two edges for the black partition stacks.



Lemma: Let c be a color of an Empty Hexasort instance $I = (G, S, t)$ such that S contains exactly four stacks $(c, h_\ell), (c, h_\ell), (c, h_s), (c, h_s)$ of color c in the given order and where $h_s < \frac{t}{2} < h_\ell$ such that $h_s + h_\ell \geq t...$

Possible Questions and Answers



- Why is sequence S not in the Venn-Diagram?
- Constant S gets polynomial time by bruteforce $O(|V|^{|S|})$.
- Polynomially bounded S gets superpolynomial.

